ĐẠI HỌC QUỐC GIA HÀ NỘI

ĐẠI HỌC KHOA HỌC TỰ NHIÊN

Khoa Toán - Cơ - Tin học



HỌC MÁY

BÁO CÁO BÀI TẬP LỚN GIỮA KỲ

Ứng dụng các phương pháp học máy trong chủ đề đánh giá sức khỏe tâm lý

Nhóm sinh viên thực hiện:

Phạm Đức Duy - 23001855

Trần Hoàng Đạt - 22001558

Giảng viên bộ môn:

TS. Cao Văn Chung

HÀ NỘI - 2024

Mục lục

Lời cảm ơn **5**

Tóm tắt **6**

Chương 1: Khái quát đề tài **7**

Chương 2: Tiền xử lý, làm sạch và phân tích dữ liệu **8**

2.1 Mô tả bộ dữ liệu 8

2.2 Tiền xử lý và làm sạch bộ dữ liệu 9

2.2.1 Tiền xử lý bộ dữ liệu 9

2.2.2 Làm sạch bộ dữ liệu 9

2.3 Phân tích bộ dữ liệu được làm sạch 10

2.3.1 Tóm tắt thông số 10

2.3.2 Biểu đồ Word Cloud 11

2.3.3 Phân tích n-grams 12

Chương 3: Trích xuất đặc trưng với TF-IDF **14**

3.1 Nội dung phương pháp 14

3.2 Ma trận Term-Document 14

3.3 Bộ dữ liệu được trích xuất bởi TF-IDF 16

3.4 Độ tương đồng Cosine 17

3.4.1 Độ tương đồng Cosine 17

3.4.2 Phân tích bộ dữ liệu TF-IDF bằng độ tương đồng Cosine 17

Chương 4: Phương pháp Latent Semantic Analysis **19**

4.1 Nội dung phương pháp 19

4.2 Khai triển kì dị rút gọn 19

4.3 Bộ dữ liệu giảm chiều bởi LSA 20

4.4 Phân tích chủ đề tiềm ẩn 21

4.4.1 Phân tích theo phần tử lớn nhất 22

4.4.2 Phân tích theo độ tương đồng Cosine 23

4.5 Trực quan hóa và Biplots 24

4.5.1 Trực quan hóa 24

4.5.2 Biplots 25

Chương 5: Phân cụm dữ liệu **27**

5.1 Khái quát quá trình phân cụm 27

5.2 Phương pháp phân cụm K-Means 27

5.2.1 Khái quát phương pháp 27

5.2.2 Hàm tổn thất 27

5.2.3 Các bước thuật toán 28

5.2.4 Trực quan kết quả phân cụm K-Means 28

5.3 Phương pháp phân cụm Gaussian Mixture Model 30

5.3.1 Khái quát phương pháp 30

5.3.2 Tổng quan mô hình và hàm mục tiêu . . . 30
5.3.3 Thuật toán Estimation-Maximization . . . 30
5.3.4 Trực quan kết quả phân cụm GMM . . . 31
Khái quát quá trình phân loại . . . 33
Chương 6: Phân loại bằng mô hình K-Nearest Neighbors . . . **34**
6.1 Nội dung phương pháp . . . 34
6.1.1 Khái quát phương pháp . . . 34
6.1.2 Hàm khoảng cách - Euclidean và Cosine . . . 34
6.1.3 Cách mô hình đưa ra dự đoán . . . 34
6.2 Áp dụng mô hình và kết quả . . . 35
6.2.1 Chạy trên bộ dữ liệu TF-IDF . . . 35
6.2.2 Chạy trên bộ dữ liệu LSA . . . 35
6.3 Đánh giá mô hình . . . 36
6.3.1 Nhận xét kết quả . . . 36
6.3.2 Mô hình tốt nhất . . . 36
6.3.3 Giải thích mô hình . . . 36
Chương 7: Phân loại bằng mô hình Random Forest . . . **37**
7.1 Nội dung phương pháp . . . 37
7.1.1 Khái quát phương pháp . . . 37
7.1.2 Tổng quan mô hình . . . 37
7.1.3 Cây quyết định . . . 37
7.1.4 Độ tinh khiết Gini . . . 38
7.1.5 Cách mô hình đưa ra dự đoán . . . 38
7.2 Áp dụng mô hình và kết quả . . . 39
7.2.1 Chạy trên bộ dữ liệu TF-IDF . . . 39
7.2.2 Chạy trên bộ dữ liệu LSA . . . 39
7.3 Đánh giá mô hình . . . 39
7.3.1 Nhận xét kết quả . . . 39
7.3.2 Mô hình tốt nhất . . . 40
7.3.3 Giải thích mô hình . . . 40
Chương 8: Phân loại bằng mô hình Logistic Regression . . . **41**
8.1 Khái quát phương pháp . . . 41
8.2 Tổng quan mô hình Multinomial Logistic Regression . . . 41
8.2.1 Tổng quan mô hình và hàm Softmax . . . 41
8.2.2 Hàm tổn thất . . . 42
8.2.3 Thuật toán Gradient Descent . . . 42
8.2.4 L2 Regularization . . . 43
8.3 Tổng quan mô hình Binary Logistic Regression . . . 44
8.4 Áp dụng mô hình và kết quả . . . 46
8.4.1 Chạy trên bộ dữ liệu TF-IDF . . . 46
8.4.2 Chạy trên bộ dữ liệu LSA . . . 46
8.5 Đánh giá mô hình . . . 46
8.5.1 Nhận xét kết quả . . . 46
8.5.2 Mô hình tốt nhất . . . 47

8.5.3 Giải thích mô hình 47
Tổng hợp kết quả phân loại 48
Chương 9: Kết luận **49**
9.1 Tóm tắt kết quả và Nhận xét chung 49
9.2 Hạn chế của bài báo cáo 49
9.3 Định hướng nghiên cứu tương lai 49
Danh sách hình minh họa và bảng **50**
Tài liệu tham khảo **51**

Lời cảm ơn

Chúng em xin gửi lời cảm ơn sâu sắc đến TS. Cao Văn Chung, người đã luôn tận tình chia sẻ kiến thức quý báu và hỗ trợ nhóm trong suốt quá trình thực hiện báo cáo này. Mặc dù không trực tiếp hướng dẫn, nhưng sự hướng dẫn tinh thần và sự hỗ trợ chuyên môn của thầy đã giúp nhóm hoàn thành công việc một cách tốt nhất.

Nhóm em cũng xin chân thành cảm ơn thầy đã truyền đạt những kiến thức quý báu, giúp chúng em phát triển kỹ năng và hiểu biết trong lĩnh vực học máy. Những kiến thức này không chỉ là nguồn động viên mà còn là cơ sở vững chắc cho sự nghiên cứu và phát triển sau này.

Một lần nữa, chúng em xin chân thành cảm ơn và kính chúc thầy mạnh khỏe và hạnh phúc trong cuộc sống, cùng tiếp tục chia sẻ kiến thức và đạt được nhiều thành công trong công việc giáo dục và nghiên cứu.

Hà Nội, Ngày 8 Tháng 12 năm 2024

Tóm tắt

Mạng xã hội đang ngày càng trở nên phổ biến, trở thành nơi người dùng dễ dàng chia sẻ cảm xúc và trạng thái tâm lý. Điều này tạo ra một nguồn dữ liệu vô cùng quý giá, góp phần phát hiện những biểu hiện ngôn ngữ bất thường có thể liên quan đến các vấn đề tâm lý. Báo cáo này tập trung phân tích và khai thác một bộ dữ liệu văn bản xoay quanh chủ đề trên, đồng thời xây dựng các mô hình phân loại nhằm giải quyết bài toán này.

TF-IDF được sử dụng làm phương pháp chính để trích xuất đặc trưng, trong khi LSA và độ tương đồng Cosine hỗ trợ phân tích cấu trúc và khám phá chủ đề của bộ dữ liệu. Các thuật toán phân cụm như K-Means và GMM được áp dụng để nhóm các mẫu văn bản có đặc điểm tương đồng. Bên cạnh đó, các mô hình K-NN, Random Forest và Logistic Regression được triển khai và thử nghiệm để giải quyết bài toán phân loại.

Mỗi phương pháp sẽ được trình bày chi tiết, từ giải thích lý thuyết đến quá trình triển khai, thực nghiệm, đánh giá và nhận xét kết quả. Toàn bộ quy trình được thực hiện với sự cân nhắc cẩn thận, phù hợp với tính nhạy cảm và phức tạp của vấn đề sức khỏe tâm lý.

Từ khóa: Sức khỏe tâm lý, Xử lý ngôn ngữ tự nhiên, Latent Semantic Analysis, K-Means, Gaussian Mixture Model, K-Nearest Neighbors, Random Forest, Logistic Regression.

Chương 1: Khái quát đề tài

Lí do chọn đề tài: Sức khỏe tâm lý là một vấn đề đang được quan tâm trong xã hội hiện đại, đặc biệt khi xu hướng cô lập và cảm giác cô đơn gia tăng, làm suy giảm chất lượng cuộc sống của nhiều người. Trong bối cảnh này, mạng xã hội đã trở thành công cụ phổ biến, giúp mọi người dễ dàng chia sẻ cảm xúc và trạng thái tâm lý, đồng thời cung cấp một nguồn dữ liệu quý giá để phân tích và hỗ trợ nâng cao sức khỏe tinh thần cho cộng đồng. Do đó, việc sớm nhận diện các dấu hiệu bất thường như lo âu hay trầm cảm là vô cùng quan trọng, nhằm kịp thời cung cấp hỗ trợ và tài nguyên cần thiết cho những người đang cần giúp đỡ.

Mục tiêu nghiên cứu: Khám phá quá trình sử dụng các phương pháp học máy và xử lý ngôn ngữ tự nhiên khi làm việc với một bộ dữ liệu văn bản. Khai thác và phân tích về cấu trúc, nội dung, chủ đề trong bộ dữ liệu. Thực nghiệm và đánh giá, so sánh các phương pháp, mô hình được áp dụng khi xét đến bản chất nhạy cảm và phức tạp của chủ đề sức khỏe tâm lý.

Cấu trúc báo cáo:

- **Chương 1: Khái quát đề tài** - Lí do chọn đề tài, mục tiêu nghiên cứu và khái quát cấu trúc của bài báo cáo.
- **Chương 2: Tiền xử lí, làm sạch và phân tích dữ liệu** - Mô tả bộ dữ liệu, tóm tắt quá trình tiền xử lí và làm sạch bộ dữ liệu, phân tích bộ dữ liệu.
- **Chương 3: Trích xuất đặc trưng với TF-IDF** - Khám phá phương pháp TF-IDF cho việc trích xuất đặc trưng từ bộ dữ liệu văn bản và phân tích bộ dữ liệu được trích xuất bởi TF-IDF.
- **Chương 4: Phương pháp Latent Semantic Analysis** - Khám phá phương pháp LSA cho việc phân tích, giảm chiều và trực quan hóa bộ dữ liệu. Phân tích bộ dữ liệu giảm chiều bởi LSA.
- **Chương 5: Phân cụm dữ liệu** - Áp dụng mô hình K-Means và Gaussian Mixture Model để phân cụm bộ dữ liệu, phân tích và trực quan hóa kết quả. So sánh hiệu quả phân cụm của hai mô hình.
- **Khái quát quá trình phân loại** - Nêu rõ các bộ dữ liệu được sử dụng, các thang đo đánh giá và quy trình phân loại chung.
- **Chương 6: Phân loại bằng mô hình K-Nearest Neighbors** - Áp dụng mô hình K-NN để phân loại. Xây dựng mô hình, thực nghiệm, đánh giá, nhận xét và giải thích kết quả.
- **Chương 7: Phân loại bằng mô hình Random Forest** - Áp dụng mô hình Random Forest để phân loại. Xây dựng mô hình, thực nghiệm, đánh giá, nhận xét và giải thích kết quả.
- **Chương 8: Phân loại bằng mô hình Logistic Regression** - Áp dụng mô hình Logistic Regression để phân loại. Xây dựng mô hình, thực nghiệm, đánh giá, nhận xét và giải thích kết quả.
- **Tổng hợp kết quả phân loại** - Tổng kết và so sánh các kết quả phân loại của từng mô hình, đưa ra đánh giá và nhận xét.
- **Chương 9: Kết luận** - Tóm tắt kết quả và đưa ra nhận xét chung, nêu rõ hạn chế của bài báo cáo và định hướng nghiên cứu tương lai.

Mã nguồn của bài báo cáo: *mental-health-corpus*.

Chương 2: Tiền xử lý, làm sạch và phân tích dữ liệu

2.1 Mô tả bộ dữ liệu

Nguồn bộ dữ liệu: Bộ dữ liệu được chia sẻ trên nền tảng Kaggle. Do những lo ngại về quyền riêng tư, có rất ít thông tin về bộ dữ liệu ngoài những gì được cung cấp tại: **Mental Health Corpus**.[3]

Mô tả bộ dữ liệu:

- Bộ dữ liệu là một bộ *Corpus* (một tập hợp các văn bản) gồm 27977 mẫu văn bản trong ngôn ngữ tiếng Anh, với hai cột: *"text"* và *"label"*.
- Cột *"text"* chứa các văn bản như bình luận hay bài đăng từ người dùng trên các nền tảng mạng xã hội.
- Cột *"label"* có giá trị 0 hoặc 1, trong đó:
  - *Giá trị 1* biểu thị văn bản có dấu hiệu bất thường, phản ánh nội dung liên quan đến sức khỏe tâm lý của người viết.
  - *Giá trị 0* đại diện cho văn bản không liên quan tới chủ đề được nói trên.
- Bộ dữ liệu có phân bổ khá cân bằng với 14139 mẫu thuộc lớp 0 (chiếm ∼50.54%) và 13838 mẫu thuộc lớp 1 (chiếm ∼49.46%).

text	label
tik tok bad annoying person bus thought never gonna give tik tok lost faith humanity	0
whats name monster remove head still walk around take head set somewhere still talk body...	0
im scared mom getting better rage dad im scared going kill broght hoping get shot late im scared life	1
im disgusted myself im disgusted self want kill whine im going isnt even bad many others much...	1
funny would former power ranger using theyre stunt man training rob ppl	0

Bảng 1: Một vài mẫu văn bản và lớp của chúng trong bộ dữ liệu

Chủ đề sức khỏe tâm lý là một lĩnh vực nhạy cảm và phức tạp, đòi hỏi sự thấu hiểu sâu sắc và sự hỗ trợ từ các chuyên gia tâm lý để đảm bảo tính chính xác và giá trị của bất kỳ nghiên cứu nào. Trong phạm vi báo cáo này, chúng tôi xin nhấn mạnh rằng các nhận định và quan điểm được trình bày chỉ phản ánh góc nhìn cá nhân, dựa trên những nỗ lực tìm hiểu và nghiên cứu. Chúng tôi luôn trân trọng tầm quan trọng của việc giữ cho báo cáo được khách quan và trung lập, đồng thời mong muốn đóng góp một cách ý nghĩa và chân thành vào việc nâng cao nhận thức về lĩnh vực này.

Bộ dữ liệu được đánh giá là *cân bằng*, với sự chênh lệch nhỏ giữa hai lớp: lớp 0 và lớp 1. Vì cột "label" chỉ bao gồm hai giá trị, bài toán phân loại trong báo cáo này được xác định là bài toán *phân loại nhị phân*.

Dữ liệu văn bản khác biệt đáng kể so với dữ liệu bảng hoặc hình ảnh do thứ tự các thành phần đầu vào đóng vai trò quan trọng. Đồng thời, kích thước đầu vào thường không cố định, và các ý nghĩa của đặc trưng trong văn bản có thể thay đổi linh hoạt, ví dụ như từ đồng nghĩa hoặc ngữ cảnh khác nhau.

Ngoài ra, các văn bản trong bộ dữ liệu thường chứa nhiều yếu tố gây nhiễu, chẳng hạn như lỗi chính tả, tiếng lóng, viết tắt, hoặc thậm chí là từ vô nghĩa — một đặc điểm phổ biến khi thu thập dữ liệu từ các nền tảng mạng xã hội. Những yếu tố này có thể là thách thức lớn, ảnh hưởng đến hiệu quả xử lý và phân tích dữ liệu văn bản.

2.2 Tiền xử lý và làm sạch bộ dữ liệu

2.2.1 Tiền xử lý bộ dữ liệu

Trong *Xử lý Ngôn ngữ Tự nhiên (NLP - Natural Language Processing)*, tiền xử lý dữ liệu là một bước nền tảng quan trọng để đảm bảo dữ liệu văn bản được theo một tiêu chuẩn nhất định. Văn bản thô thường chứa lỗi chính tả, ký tự đặc biệt, từ dư thừa, hoặc các yếu tố gây nhiễu khác, ảnh hưởng đến hiệu suất của các mô hình.[20]

Các bước tiền xử lý dữ liệu được thực hiện bao gồm:

- Loại bỏ dấu câu, ký tự đặc biệt, chữ số, các yếu tố thẻ HTML và các đường liên kết (chứa "http", "www",...).
- Chuyển đổi mọi ký tự chữ cái in hoa về in thường.
- Chuẩn hóa các khoảng trắng về một dấu cách.
- Loại bỏ các mẫu trống, chỉ chứa khoảng trắng hoặc bị trùng lặp.

2.2.2 Làm sạch bộ dữ liệu

Sau bước tiền xử lý, quá trình làm sạch dữ liệu thường mang tính đặc thù hơn và độc đáo với từng bộ Corpus. Những công việc này bao gồm loại bỏ các từ hoặc yếu tố có thể gây sai lệch dữ liệu, sửa lỗi chính tả phổ biến, và các điều chỉnh khác. Việc làm sạch thường được dựa trên động lực phân tích chi tiết bộ dữ liệu sau khi đã được tiền xử lý. Trước tiên, ta cần hiểu một số thuật ngữ quan trọng trong NLP:

- *Tokens*: Đặc trưng mang thông tin từ văn bản. Trong bài báo cáo này, Token được định nghĩa là một chuỗi ký tự được phân tách bởi khoảng trắng, hay nói cách khác là một từ.
- n-*grams*: Một tập hợp liên tiếp của n Tokens. Ví dụ, nếu coi "not" và "happy" là các Tokens riêng lẻ (*Unigram* / 1-gram), thì "not happy" là một *Bigram* (2-gram).[27]
- *Lemmatization*: Kỹ thuật đưa một từ về dạng gốc (*lemma*) dựa trên ý nghĩa và ngữ pháp. Lemmatization tận dụng quy tắc ngữ pháp và các từ điển để giữ lại ý nghĩa đầy đủ của từ. Ví dụ: "rocks" trở thành "rock", "better" chuyển thành "good".[22,27]
- *Stop-words*; Các Tokens phổ biến nhưng không mang nhiều ý nghĩa ngữ cảnh, như "the", "is", "and", "of",... Việc loại bỏ Stop-words giúp mô hình tập trung vào các từ có ý nghĩa quan trọng hơn và có thể giúp phân biệt các văn bản với nhau tốt hơn.[27]

Dưới đây là **tóm tắt các bước làm sạch bộ dữ liệu được thực hiện**, kèm theo động lực và cơ sở cho từng bước:

- Áp dụng Lemmatization và loại bỏ Stop-words: Chuẩn hóa Tokens về dạng gốc và loại bỏ các từ không mang nhiều ý nghĩa ngữ cảnh để tăng tính đồng nhất của dữ liệu.
- Loại bỏ Tokens quá dài hoặc quá ngắn: Xóa các Tokens có độ dài lớn hơn 17 ký tự hoặc nhỏ hơn 3 ký tự, ngoại trừ một số Tokens như "i", "im", "go", "do" do độ quan trọng của chúng đối với lớp 1.[41]
- Xử lý Tokens lặp lại không hợp lý: Loại bỏ Tokens xuất hiện với tần suất bất thường qua phân tích n-grams (với $n = 2, 3, 4, 5$).
- Loại bỏ Tokens hiếm gặp: Xóa các Tokens có tần suất xuất hiện dưới 5 lần do chúng thường là lỗi chính tả, từ lóng hoặc từ không có giá trị ngữ nghĩa.
- Giới hạn số lượng Tokens của mỗi văn bản: Chỉ giữ lại các văn bản có số lượng Tokens trong khoảng $[10, 400]$ để đảm bảo lượng thông tin phù hợp.

2.3 Phân tích bộ dữ liệu được làm sạch

2.3.1 Tóm tắt thông số

Sau quá trình tiền xử lý và làm sạch, bảng dưới đây cho thấy sự thay đổi giữa bộ Corpus gốc và bộ Corpus được làm sạch:

Corpus	Category	Total Samples	Total Tokens	Unique Tokens
Original Corpus	Class 0	14139 (∼50.54%)	669862	48965
	Class 1	13838 (∼49.46%)	1337568	42088
	Entire Corpus	27977 (100.00%)	2007430	72559
Cleaned Corpus	Class 0	10592 (∼45.58%)	526890	13657
	Class 1	12648 (∼54.42%)	1033603	12088
	Entire Corpus	23240 (100.00%)	1560493	14470

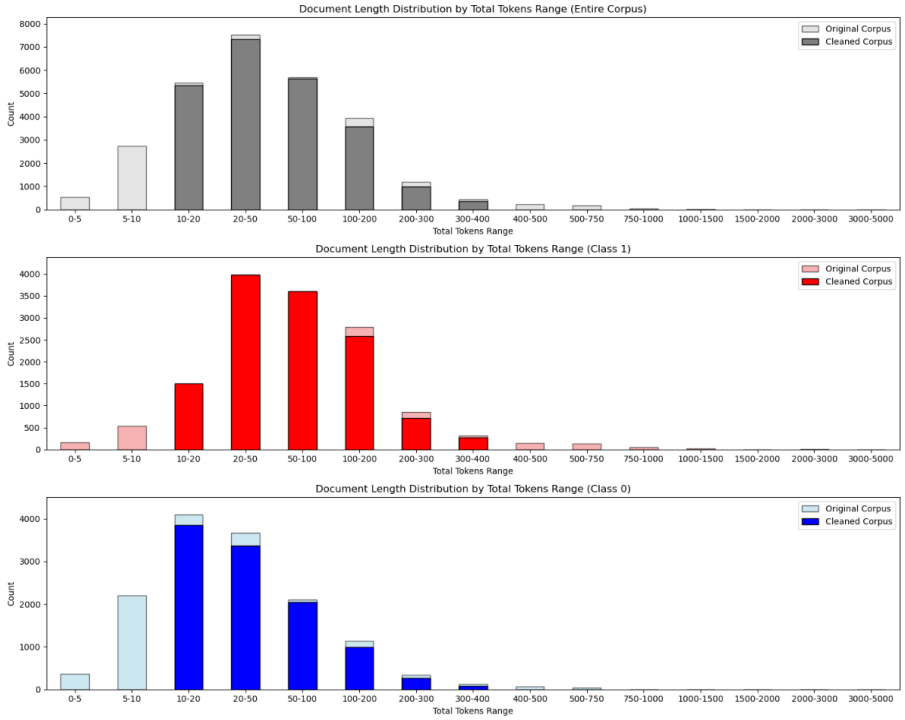
Bảng 2: Thông số bộ Corpus gốc và bộ Corpus được làm sạch

Trong đó **Total Samples** biểu thị số mẫu văn bản, **Total Tokens** và **Unique Tokens** lần lượt là tổng số Tokens và số lượng Tokens độc nhất trên cả một Corpus. Sau quá trình làm sạch:

- Số Tokens độc nhất giảm từ 72559 xuống còn 14470 (giảm ∼80.06%).
- Tổng số Tokens giảm từ 2007430 xuống 1560493 (giảm ∼22.26%).

Điều này cho thấy việc làm sạch đã loại bỏ đáng kể nhiễu nhưng vẫn giữ lại phần lớn thông tin cần thiết.

Hơn nữa, bộ Corpus sạch giảm khoảng 4737 mẫu so với Corpus bộ gốc, chủ yếu bị loại bỏ do bước chỉ giữ lại các mẫu có tổng Tokens nằm trong khoảng $[10, 400]$. Đó có thể là các mẫu văn bản quá ngắn hoặc quá dài, không mang lại nhiều ý nghĩa cho mục tiêu của báo cáo.



Hình 1: Tần suất các mẫu có tổng Tokens trong các khoảng khác nhau

Chú ý: Ta sẽ chỉ sử dụng bộ dữ liệu được làm sạch và gọi là **bộ Corpus sạch** trong suốt báo cáo kể từ đây.

2.3.2 Biểu đồ Word Cloud

Một bước quan trọng trong phân tích các bộ Corpus là xác định các Tokens phổ biến nhất và ít phổ biến nhất. Kết hợp với *biểu đồ Word Cloud* - biểu đồ trực quan hiển thị các Tokens theo kích thước tương ứng với tần suất — giúp chúng ta dễ dàng phân tích và hiểu rõ dữ liệu hơn.

Dưới đây là kết quả trích xuất các Tokens phổ biến nhất và 1000 Tokens ít phổ biến nhất trong bộ Corpus sạch, với tập văn bản thuộc lớp 0 và lớp 1. Tổng cộng có 6 biểu đồ Word Cloud được tạo.



Hình 2: Các biểu đồ Word Cloud được tạo ra trong quá trình phân tích

Với các biểu đồ Word Cloud của các Tokens phổ biến nhất (3 biểu đồ bên trái), hầu hết Tokens xuất hiện đồng đều trên toàn bộ Corpus, lớp 0 và lớp 1. Các Tokens như "feel" và "life" chiếm diện tích lớn, gợi ý nhiều văn bản thuộc lớp 1 liên quan đến cảm xúc hoặc cuộc sống. Đáng chú ý, các Tokens như "film" và "movie" nổi bật trong lớp 0, chỉ ra rằng nhiều văn bản thuộc lớp này có chủ đề liên quan đến phim ảnh.

Ở biểu đồ Word Cloud của 1000 Tokens ít phổ biến nhất (3 biểu đồ bên phải), dù đã làm sạch dữ liệu, các từ viết sai chính tả, từ vô nghĩa hoặc tiếng lóng vẫn xuất hiện, cho thấy rằng nhiễu trong văn bản chỉ có thể được giảm thiểu đến một mức độ nhất định.

2.3.3 Phân tích n-grams



Hình 3: Kết quả phân tích n-grams

Kết quả phân tích n-grams, với $n = 2, 3, 4, 5$ (tương ứng với *Bigrams, Trigrams, Tetragrams* và *Pentagrams*), mang lại cái nhìn sâu sắc về các mẫu ngôn ngữ đặc trưng trong hai lớp dữ liệu. So với việc chỉ xét các từ đơn (*Unigrams*), phân tích n-grams giúp làm rõ hơn các chủ đề và ngữ cảnh trong văn bản, đặc biệt khi nội dung văn bản có sự mơ hồ hoặc thiếu tính cấu trúc.

Đối với lớp 0, các n-grams cho thấy sự xuất hiện lặp lại của những cụm từ ít mang ý nghĩa hoặc không rõ ngữ cảnh, như "water water water" hoặc "love love love love." Điều này cho thấy một phần dữ liệu trong lớp này có xu hướng chứa những mẫu lặp không hợp lý, có thể xuất phát từ các văn bản mang tính không nghiêm túc hoặc dư thừa. Đáng chú ý, trong các biểu đồ n-grams với $n \geq 3$, sự lặp lại tăng lên rõ rệt, phản ánh những đặc điểm phổ biến nhưng không thực sự giá trị trong việc hiểu chủ đề sâu hơn. Tuy nhiên, việc giữ lại các Tokens này trong quá trình phân tích nhằm đảm bảo tính trung thực, phản ánh đúng bản chất của dữ liệu thực tế.

Ngược lại, các n-grams của lớp 1 thể hiện rõ sự liên quan đến sức khỏe tâm lý và cảm xúc tiêu cực, với các cụm từ như "feel like im," "want die want," và "please help please." Những cụm từ này làm nổi bật chủ đề tự hại, sự mệt mỏi, và nhu cầu được giúp đỡ, là đặc trưng của lớp 1. Sự xuất hiện này minh chứng rằng phân tích n-grams không chỉ giúp làm rõ nội dung mà còn phát hiện các biểu hiện ngôn ngữ đặc thù mà phân tích từ đơn hoặc Word Cloud khó có thể cung cấp.

Tóm lại, phân tích n-grams cho thấy hiệu quả trong việc khám phá các mẫu ngôn ngữ tiềm ẩn, đặc biệt khi dữ liệu đến từ các nguồn không chuẩn hóa như mạng xã hội. Trong lớp 0, các cụm từ lặp phản ánh đặc điểm nhiễu và sự đa dạng ngôn ngữ, trong khi lớp 1 làm nổi bật các cụm từ mang ý nghĩa cảm xúc và ngữ cảnh cụ thể. Như vậy, quá trình tiền xử lý và làm sạch đã được thực hiện với mức độ hợp lý nhằm giữ nguyên giá trị của dữ liệu và đảm bảo không loại bỏ quá nhiều các thông tin quan trọng.

Chương 3: Trích xuất đặc trưng với TF-IDF

3.1 Nội dung phương pháp

TF-IDF (Term Frequency - Inverse Document Frequency) là một phương pháp phổ biến trong lĩnh vực NLP, dùng để trích xuất đặc trưng từ văn bản. Phương pháp này được áp dụng lên các tập văn bản (Corpus), nhằm **vector hóa** các tài liệu trong đó.

Kết quả của quá trình TF-IDF là các vector có cùng độ dài, trong đó mỗi phần tử được gọi là **trọng số TF-IDF**, thể hiện *mức độ quan trọng* của một từ (token) đối với một tài liệu cụ thể trong toàn bộ tập văn bản. Phương pháp này giúp chuyển đổi một Corpus thành một bảng dữ liệu, trong đó các hàng đại diện cho các vector văn bản chứa trọng số TF-IDF, và các cột tương ứng với các Tokens được coi là đặc trưng trong tập dữ liệu.

TF-IDF là một kỹ thuật đơn giản, dễ áp dụng và hiệu quả trong việc vector hóa văn bản. Ngoài ra, phương pháp này còn có khả năng **giảm nhiễu**, giúp loại bỏ nhiều yếu tố phức tạp của ngôn ngữ tự nhiên. Tuy nhiên, do chỉ tập trung vào tần suất xuất hiện, TF-IDF không lưu giữ bất kỳ thông tin nào về **ngữ nghĩa** hay **ngữ cảnh** của từ trong tài liệu.

3.2 Ma trận Term-Document

Khi được áp dụng lên một bộ Corpus để trích xuất đặc trưng, TF-IDF sẽ trả về một ma trận được gọi là **Ma trận Term-Document (Ma trận TD)**. Định nghĩa như sau:

Cho một bộ Corpus $\mathcal{D} = \{D_1, D_2, \ldots, D_N\}$ gồm N văn bản và tập từ vựng $T = \{t_1, t_2, \ldots, t_n\}$, ta xây dựng ma trận trọng số:

$$W_n = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{N1} & w_{N2} & \cdots & w_{Nn} \end{bmatrix} \in \mathbb{R}^{N \times n}$$

Trong đó mỗi phần tử w_{ij}, gọi là **trọng số TF-IDF** được tính theo công thức:[27]

$$w_{ij} = \text{tf}_{ij} \times \left(\log \left(\frac{N+1}{\text{df}_j + 1} \right) + 1 \right)$$

Giải thích từng thành phần:

- **Term Frequency (TF):**

$$\text{tf}_{ij} = \frac{\text{số lần xuất hiện của } t_j \text{ trong } D_i}{\text{tổng số tokens trong } D_i}$$

  Phần TF phản ánh tần suất tương đối của từ t_j trong văn bản D_i.

- **Document Frequency (DF):**

$$\text{df}_j = \text{số văn bản trong } \mathcal{D} \text{ chứa } t_j$$

  Phần DF thể hiện mức độ phổ biến của từ t_j trên toàn bộ tập văn bản $\mathcal{D}$.

- **Inverse Document Frequency (IDF):**

$$\text{idf}_j = \log \frac{N+1}{\text{df}_j + 1} + 1$$

Phần IDF giúp giảm trọng số của các từ phổ biến xuất hiện trong hầu hết các văn bản, đồng thời tăng trọng số cho các từ hiếm. Việc cộng hằng số 1 vào tử số và mẫu số nhằm tránh chia cho 0 khi $\text{df}_j = 0$.

Nhân phần TF và phần IDF với nhau, ta được trọng số TF-IDF, cũng là lý do vì sao phương pháp này có tên là TF-IDF. Sau đây ta sẽ phân tích sâu hơn về ý nghĩa của trọng số w_{ij}:

- **Độ quan trọng của từ t_j trong văn bản D_i:** Trọng số w_{ij} là sản phẩm của TF và IDF, kết hợp tần suất xuất hiện của từ trong văn bản cụ thể với mức độ hiếm của từ trên toàn tập văn bản.
  - Nếu t_j xuất hiện nhiều trong D_i và ít phổ biến trong tập $\mathcal{D}$, w_{ij} sẽ cao.
  - Nếu t_j phổ biến trong toàn bộ tập $\mathcal{D}$, w_{ij} sẽ thấp, bất kể tần suất của nó trong D_i.
- **Khả năng phân biệt tài liệu:** TF-IDF làm nổi bật các từ có tính phân biệt cao, giúp nhận diện nội dung đặc trưng của từng văn bản.
- **Giảm ảnh hưởng của các từ không quan trọng:** Những từ phổ biến như từ nối (e.g., *and*, *or*) thường có IDF thấp, dẫn đến trọng số TF-IDF nhỏ, từ đó giảm tác động của chúng trong quá trình phân tích.

Bảng dưới đây minh họa ý nghĩa của trọng số TF-IDF trong các trường hợp kết hợp khác nhau giữa TF và IDF:

Giá trị TF	Giá trị IDF	Trọng số TF-IDF	Ý nghĩa
Cao	Cao	Cao	Token xuất hiện nhiều trong văn bản và khá hiếm trong toàn bộ Corpus. Nó rất quan trọng trong việc phân biệt văn bản này với các văn bản khác.
Cao	Thấp	Thấp	Token xuất hiện nhiều trong văn bản nhưng cũng phổ biến trong toàn bộ Corpus. Nó không giúp hoặc giúp ít cho việc phân biệt văn bản này với các văn bản khác.
Thấp	Cao	Trung bình	Token ít xuất hiện trong văn bản và rất hiếm trong toàn bộ Corpus. Nó vẫn có thể mang lại giá trị phân biệt.
Thấp	Thấp	Thấp	Token ít xuất hiện trong văn bản và phổ biến trong toàn bộ Corpus. Nó không giúp hoặc giúp ít cho việc phân biệt văn bản này với các văn bản khác.

Bảng 3: Giải thích về trọng số TF-IDF với các kết hợp giá trị TF và IDF khác nhau

Các trọng số TF-IDF có thể được hiểu là biểu thị mức độ quan trọng của các từ *(term)* trong mối quan hệ với các văn bản cụ thể *(document)*, lần lượt theo các hàng và cột của ma trận. Do đó ma trận W_n được gọi là *ma trận Term-Document*. Dễ thấy, các ma trận TD thường rất thưa thớt, bởi trong một văn bản D_i chỉ có một tập nhỏ các từ vựng xuất hiện thực tế $T^{(i)}$, mà tập này là một tập con của toàn bộ từ vựng T, tức $T^{(i)} \subset T$.

Trong thực tế, tập từ vựng $T = \{t_1, t_2, \ldots, t_n\}$ của một corpus thường rất lớn. Để giảm kích thước, ta có thể chọn một tập con $T_s = \{t_1, t_2, \ldots, t_s\} \subset T$ với $s << n$. Tập con này được xác định bằng cách sắp xếp các từ theo tổng giá trị đặc trưng lớn nhất, giúp giảm nhiễu bằng cách loại bỏ các đặc trưng ít giá trị trong việc phân biệt văn bản.

Khi đó, ma trận $W_s \in \mathbb{R}^{N\times s}$ được xây dựng bằng phương pháp TF-IDF trên bộ dữ liệu $\mathcal{D} = \{D_1, D_2, \ldots, D_N\}$ sẽ tạo ra N Vector văn bản $D_i \in \mathbb{R}^s$. Các vector này được chuẩn hóa để loại bỏ ảnh hưởng của độ dài văn bản trong quá trình so sánh:

$$D_i = \left\{ \frac{w_{i1}}{\|w_{i1}\|_2}, \frac{w_{i2}}{\|w_{i2}\|_2}, \ldots, \frac{w_{is}}{\|w_{is}\|_2} \right\}; \quad \|D_i\|_2 = 1$$

Các vector kết quả có một số đặc điểm quan trọng: không chứa giá trị âm (do TF và IDF luôn không âm), thưa, và được chuẩn hóa.

3.3 Bộ dữ liệu được trích xuất bởi TF-IDF

Khi áp dụng TF-IDF lên một bộ Corpus, giá trị TF được tính trực tiếp cho từng Token trong từng văn bản, trong khi giá trị IDF được xác định dựa trên toàn bộ Corpus. Do đó, khi xử lý dữ liệu mới, TF sẽ phụ thuộc vào văn bản đầu vào, trong khi IDF của các Tokens vẫn giữ nguyên hoặc cho bằng 0 nếu Tokens không xuất hiện trong Corpus gốc. Như vậy, các từ không xuất hiện trong Corpus sẽ được cho trọng số TF-IDF bằng 0.

Cơ chế này nhấn mạnh tầm quan trọng của việc lựa chọn s sao cho vừa giảm nhiễu hiệu quả vừa duy trì tính linh hoạt của tập từ vựng T_s khi gặp dữ liệu mới.

Sau khi áp dụng TF-IDF lên bộ Corpus sạch, ta thu được dữ liệu đầu ra được biểu diễn bởi ma trận $W_s \in \mathbb{R}^{23240\times s}$. Ta có thể quyết định giá trị s dựa vào quá trình phân tích các Tokens nằm cuối tập từ vựng $T_s = \{t_1, t_2, \ldots, t_s\}$ với các giá trị s khác nhau:

s	**Bottom Tokens**
2000	income, officer, manager, appears, threatened, tone, academic, studio, patient, regardless, advantage, decides, richard, atmosphere, comic, divorced, coworkers,...
3000	maddock, clack, brice, intended, narrative, merely, naive, columbo, village, provides, united, charlie, allows, customer, flesh, inspired, receiving, absence, desert, settle,...
3500	maddock, haa, clack, ameno, brice, lick, wealthy, delivers, alex, hitchcock, anne, jason, historical, passionate, associate, attending, importance, revealed, grabbed, cling,...
4500	clickclick, hoo, maddock, haa, dor, ameno, clack, brice, konf, sauce, custer, maria, bruno, hee, sidney, lucy, carpenter, cinderella, bourne, stephen,...
6000	friesmy, clinker, scutter, clickclick, haa, hoo, maddock, dor, gaur, ameno, clack, brice, axolotl, bjvgze, rdogeprime, konf, kolchak, monkees, tenant, fay,...

Bảng 4: Các Tokens có tổng phần tử nhỏ nhất trong các tập từ vựng của TF-IDF với các giá trị s khác nhau

Có thể thấy kể từ $s = 3500$, phần lớn các token ở cuối tập T_s là các từ viết sai chính tả, tên riêng hoặc không mang nhiều ý nghĩa. Vì vậy, ta chọn $s = 3500$, tạo ra ma trận đặc trưng $W_{3500} \in \mathbb{R}^{23240\times 3500}$, ta gọi là **bộ dữ liệu TF-IDF** sử dụng xuyên suốt báo cáo này kể từ đây.

Để kiểm chứng các đặc điểm của bộ dữ liệu, ta chọn một số vector văn bản trong W_{3500} để phân tích:

Doc. Index	Zeros	Non-Zero	Magnitude	Sign Distribution
11456	3473	27	1.0000	100% Non-Negative
1288	3488	12	1.0000	100% Non-Negative
5535	3489	11	1.0000	100% Non-Negative
12657	3492	8	1.0000	100% Non-Negative
8603	3482	18	1.0000	100% Non-Negative

Bảng 5: Đặc điểm của một vài Vector văn bản tạo bởi TF-IDF

- Ma trận W_{3500} rất thưa, với phần lớn phần tử có giá trị bằng 0.
- Các vector không chứa phần tử âm do phần TF và IDF luôn không âm.
- Mỗi vector được chuẩn hóa theo chuẩn Euclidean, tức là chuẩn của các vector bằng 1.

Hiểu rõ những đặc điểm này không chỉ giúp lựa chọn các phương pháp xử lý phù hợp mà còn đóng vai trò quan trọng trong việc tối ưu hóa hiệu suất và độ chính xác của các mô hình.

3.4 Độ tương đồng Cosine

3.4.1 Độ tương đồng Cosine

Cosine Similarity hay *Độ tương đồng Cosine* chính là Cosine của góc giữa 2 Vector khác không, cùng nằm trong $\mathbb{R}^n$. Với 2 Vector bất kì u và v $(u, v \neq \vec{0} \mid u, v \in \mathbb{R}^n)$, độ tương đồng Cosine được định nghĩa như sau:

$$S_c(u, v) = \cos\theta = \frac{u \cdot v}{\|u\| \cdot \|v\|} \in [-1; 1]$$

Trong đó:

- $u \cdot v$ là tích vô hướng của hai Vector u và v.
- $\|u\|$ và $\|v\|$ là độ dài chuẩn của các Vector u và v.

Giá trị của S_c có các ý nghĩa cụ thể:

- $S_c = 1$ khi u và v cùng hướng.
- $S_c = -1$ khi u và v ngược hướng.
- $S_c = 0$ khi hai Vector này vuông góc (trực giao) với nhau.

Chú ý: Đối với các Vector được chuẩn hóa, $S_c(u, v) = u \cdot v$.

3.4.2 Phân tích bộ dữ liệu TF-IDF bằng độ tương đồng Cosine

Với ma trận TD W_{3500}, ta có thể áp dụng độ tương đồng Cosine lên các Vector văn bản (các hàng của ma trận) để phân tích kết quả. Cụ thể, giá trị $S_c(D_i, D_j)$ càng cao cho thấy 2 văn bản D_i và D_j càng "tương đồng" với nhau. Lưu ý rằng do đặc điểm không âm và được chuẩn hóa của các Vector này, giá trị của $S_c(D_i, D_j)$ chỉ có thể nằm trong khoảng $[0, 1]$.

Doc. Index	**Class**	S_c	**Doc. Index**	**Class**	S_c	**Doc. Index**	**Class**	S_c
15464 (Query)	0	-	1841 (Query)	1	-	23176 (Query)	1	-
19288	0	0.492	23026	1	0.439	18270	0	0.361
11111	0	0.426	9854	1	0.375	1713	0	0.285
7077	0	0.402	1753	1	0.369	8827	1	0.279
16470	0	0.371	18460	1	0.358	12184	0	0.256
10582	0	0.366	263	1	0.355	9848	1	0.243

Bảng 6: Các văn bản có độ tương đồng Cosine lớn nhất với các văn bản được chọn (TF-IDF)

Độ tương đồng Cosine chứng tỏ hiệu quả cao trong việc xác định các văn bản có độ tương đồng cao, thậm chí những văn bản này còn thường thuộc cùng một lớp với văn bản được xét.

Hơn nữa, với ma trận W_{3500}, độ tương đồng Cosine cũng có thể được áp dụng cho các Vector đặc trưng (các cột của ma trận). Tuy nhiên, ý nghĩa của những kết quả thu được từ phép đo này thường không phản ánh chính xác về "độ tương đồng" giữa các Vector đặc trưng. Dưới đây là bốn ví dụ với bốn Tokens được chọn lọc để minh họa cho quá trình này:

Tokens	**"suicidal"**	**"depression"**	**Tokens**	**"movie"**	**"film"**
thought	0.3086	-	character	-	0.2833
know	0.1934	-	great	0.2632	-
im	0.1856	0.1825	scene	0.2584	0.3011
ideation	0.1846	-	actor	0.2575	0.2372
feel	0.1770	0.2037	film	0.2391	0.2391
feeling	0.1750	-	story	0.2390	0.2292
ive	0.1694	0.1767	plot	0.2296	0.2296
anxiety	-	0.2359	-	-	-
year	-	0.1991	-	-	-
life	-	0.1908	-	-	-
like	-	0.1718	-	-	-

Bảng 7: Các Tokens có độ tương đồng Cosine lớn nhất với các Tokens được chọn lọc (TF-IDF)

Các giá trị trong bảng trên thể hiện kết quả của độ tương đồng Cosine S_c giữa các Vector đặc trưng đại diện cho các Tokens. Nhìn chung, các Tokens có độ tương đồng cao không thực sự đồng nghĩa hay sát nghĩa, mà mối quan hệ giữa chúng chủ yếu phản ánh sự "đồng xuất hiện" trong các văn bản. Điều này nhấn mạnh nhược điểm chính của TF-IDF, đó là bỏ qua ngữ nghĩa và ngữ cảnh, chỉ tập trung vào tần suất xuất hiện.

Mặc dù vậy, các kết quả vẫn mang đến những gợi ý giá trị về chủ đề chính trong dữ liệu. Cụ thể, các Tokens có giá trị S_c cao với "suicidal" và "depression" gợi ý về các chủ đề như tự hại, trầm cảm. Tương tự, nhóm Tokens liên quan đến "movie" và "film" thể hiện rõ chủ đề phim ảnh, điện ảnh. Những mối liên hệ này cho thấy tính gắn kết và cấu trúc hợp lý của dữ liệu, đồng thời củng cố những phân tích và phát hiện của chúng ta từ quá trình xử lý và làm sạch bộ Corpus.

Chương 4: Phương pháp Latent Semantic Analysis

4.1 Nội dung phương pháp

Phân tích Ngữ nghĩa Tiềm ẩn (LSA - Latent Semantic Analysis) là một phương pháp trong NLP, chuyên dùng để phân tích và giảm chiều dữ liệu. LSA được áp dụng trên ma trận Term-Document để khám phá một *không gian chủ đề tiềm ẩn* trong bộ Corpus. Những chủ đề này có thể được coi là các thành phần cơ bản, đại diện cho cấu trúc của ma trận TD đại diện cho bộ Corpus.

LSA hoạt động dựa trên *Truncated Singular Value Decomposition (Truncated SVD)* hay *Khai triển Kì dị Rút gọn*, một phương pháp phân rã ma trận trong đại số tuyến tính. Bằng cách áp dụng Truncated SVD lên ma trận TD, LSA cho phép phân tích và trực quan hóa dữ liệu trong không gian chủ đề tiềm ẩn, từ đó phát hiện mối quan hệ giữa các văn bản và các Tokens thông qua những chủ đề này.

Không chỉ giúp khám phá các chủ đề tiềm ẩn, LSA còn hỗ trợ giảm chiều dữ liệu và loại bỏ nhiễu hiệu quả. Tuy nhiên, do vẫn dựa trên ma trận TD, không gian chủ đề tiềm ẩn chủ yếu phản ánh tần suất xuất hiện của các Tokens, mà chưa hiểu sâu về ngữ nghĩa hay ngữ cảnh. Thêm vào đó, việc giải thích các chủ đề tiềm ẩn không phải lúc nào cũng rõ ràng.

4.2 Khai triển kì dị rút gọn

Khai triển Kì dị (SVD - Singular Value Decomposition) cho rằng bất kỳ ma trận $A \in \mathbb{R}^{m \times n}$ nào cũng có thể được phân rã thành ba thành phần như sau:

$$A = U\Sigma\,(V)^{\top}$$

Trong đó:

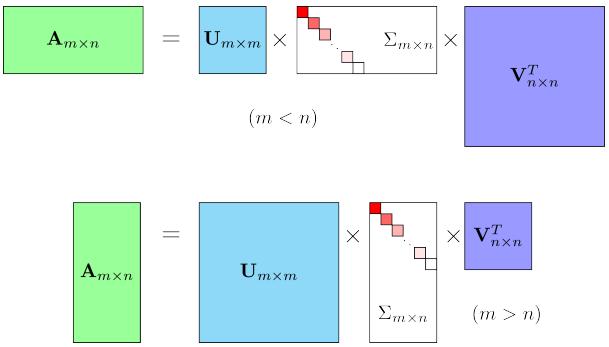
- $U \in \mathbb{R}^{m \times m}$ và $V^{\top} \in \mathbb{R}^{n \times n}$ lần lượt là các ma trận trực chuẩn chứa các Vector kì dị trái và Vector kì dị phải của A
- $\Sigma \in \mathbb{R}^{m \times n}$ là ma trận đường chéo chứa các giá trị kì dị trên đường chéo chính.

Giả sử $r = \text{rank}(A) \leq \min(m, n)$, khi đó ma trận $\Sigma_{m \times n}$ có dạng:

$$\Sigma = \begin{bmatrix} D_\sigma & 0 & \cdots & 0 \\ 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 \end{bmatrix} \quad ; \quad D_\sigma \in \mathbb{R}^{r \times r}$$

Trong đó, các phần tử trên đường chéo chính của ma trận D_σ là các giá trị kì dị, được sắp xếp theo thứ tự giảm dần: $\sigma_1 \geq \sigma_2 \geq \ldots \geq \sigma_r \geq 0$.

Dưới đây là một ví dụ trực quan minh họa kết quả của SVD đối với một ma trận $A \in \mathbb{R}^{m \times n}$ bất kì:



Hình 4: Trực quan kết quả của SVD

SVD có thể được hiểu là quá trình tìm kiếm một *không gian tiềm ẩn* gồm r thành phần, trong đó mối quan hệ giữa các hàng và cột của A được biểu diễn thông qua các ma trận $U\Sigma$ và $\Sigma V^\top$. Các giá trị kỳ dị trong Σ phản ánh *mức độ quan trọng* của từng thành phần trong không gian tiềm ẩn.

Việc tính toán SVD trên các tập dữ liệu lớn thường rất tốn kém và không hiệu quả. Do đó, **Truncated SVD** là một phương pháp phổ biến hơn. Với ma trận $A \in \mathbb{R}^{m\times n}$, Truncated SVD được mô tả như sau:

$$A \approx A_k = U_k \Sigma_k V_k^\top$$

Trong đó:

- $U_k \in \mathbb{R}^{m\times k}$, $V_k^\top \in \mathbb{R}^{k\times n}$ là các ma trận trực chuẩn theo chiều k.
- $\Sigma_k \in \mathbb{R}^{k\times k}$ là ma trận đường chéo chứa các giá trị kỳ dị $\sigma_1 \geq \sigma_2 \geq \ldots \geq \sigma_k \geq 0$.

Truncated SVD cung cấp ma trận xấp xỉ thiểu hạng tối ưu của A theo *định lý Eckart-Young-Mirsky*:[14]

$$A_k = \arg\min_{A_k} \|A - A_k\|_F \quad ; \quad \|\cdot\|_F : \text{Chuẩn Forbenious.}$$

Nếu ta coi ma trận $A \in \mathbb{R}^{m\times n}$ là 1 bộ dữ liệu bảng với m mẫu và n đặc trưng, Truncated SVD giúp chúng ta giảm chiều bộ dữ liệu về còn k đặc trưng. Khi đó không gian tiềm ẩn gồm k thành phần, các ma trận $U_k\Sigma_k \in \mathbb{R}^{m\times k}$ và $\Sigma_k V_k^\top \in \mathbb{R}^{k\times n}$ lần lượt là các ma trận chứa các Vector đại diện cho các mẫu và các đặc trưng trong không gian tiềm ẩn đó.

Khác với SVD, vốn được tính toán để tái tạo hoàn chỉnh ma trận A, Truncated SVD thường áp dụng các phương pháp tính lặp như Randomized SVD để tìm A_k là một ma trận xấp xỉ thiểu hạng tối ưu của A.

4.3 Bộ dữ liệu giảm chiều bởi LSA

Để áp dụng LSA lên bộ corpus sạch, ta thực hiện Truncated SVD trên ma trận TF-IDF W_{3500}, thu được các thành phần $U_k\Sigma_k$, $\Sigma_k V_k^\top$, lần lượt đại diện cho các Vector văn bản và các Vector đặc trưng trong không gian chủ đề tiềm ẩn với k chủ đề. Đường chéo chính của Σ_k chứa các giá trị thể hiện mức độ quan trọng của các chủ đề này.

Việc chọn k tương đương với quyết định số chủ đề tiềm ẩn có trong Corpus. Thông thường, k nằm trong khoảng 5 - 300, tùy thuộc vào kích thước của một bộ Corpus.[39] Để cân bằng giữa độ bao quát và hiệu quả giảm chiều, ta chọn $k = 100$. Điều này giúp giảm ma trận TF-IDF từ 3500 đặc trưng xuống còn 100 đặc trưng.

Như vậy, bộ dữ liệu sau khi giảm chiều là $U_{100}\Sigma_{100} \in \mathbb{R}^{23240\times 100}$, được gọi là **bộ dữ liệu LSA** xuyên suốt báo cáo kể từ đây.

Ta xét một số Vector văn bản trong bộ dữ liệu LSA nhằm kiểm chứng một vài đặc điểm:

Doc. Index	Zeros	Non-Zero	Magnitude	Sign Distribution
11456	0	100	0.3830	51% Negative, 49% Non-Negative
1288	0	100	0.4489	50% Negative, 50% Non-Negative
5535	0	100	0.2341	44% Negative, 56% Non-Negative
12657	0	100	0.3460	51% Negative, 49% Non-Negative
8603	0	100	0.2864	53% Negative, 47% Non-Negative

Bảng 8: Đặc điểm của một số Vector văn bản trong bộ dữ liệu LSA

Như vậy, từ ma trận TD W_{3500} vốn rất thưa và có các hàng được chuẩn hóa, bộ dữ liệu LSA được biểu diễn bởi ma trận $U_{100}\Sigma_{100}$, một ma trận dày đặc hơn, trong đó các hàng (các Vector văn bản) chứa cả giá trị âm và không còn được chuẩn hóa.

Bộ dữ liệu LSA cho phép thực hiện phân tích sâu hơn nhờ không gian chủ đề tiềm ẩn. Điều này vừa củng cố các kết quả trước (như cảm xúc, đời sống, phim ảnh, v.v.), vừa giúp khám phá những chủ đề mới trong bộ Corpus. Đáng chú ý, trong ngôn ngữ tự nhiên và nội dung từ mạng xã hội, các chủ đề thường đan xen với phân bổ tầm quan trọng nhất định.

Ngoài ra, việc sử dụng LSA giảm xuống 2 hoặc 3 chiều còn hỗ trợ trực quan hóa dữ liệu qua biểu đồ *Scatter Plot*, giúp ta dễ dàng nhận biết sự phân bố của các văn bản trong không gian thấp chiều. Hơn nữa, biểu đồ *Biplot* có thể minh họa đồng thời cả mẫu và đặc trưng, cung cấp một góc nhìn toàn diện hơn.

4.4 Phân tích chủ đề tiềm ẩn

Do bản chất "tiềm ẩn" của chúng, các chủ đề này thường khó giải thích hơn so với các chủ đề cụ thể trong văn bản, chẳng hạn như thể thao, khoa học hay chính trị là các chủ đề rõ ràng và rời rạc. Do đó, việc sử dụng các phương pháp phân tích hợp lý là cần thiết để phỏng đoán ý nghĩa của các chủ đề này.

Để khám phá mối quan hệ giữa văn bản và các Tokens trong không gian chủ đề tiềm ẩn, hai cách tiếp cận chính được áp dụng:

1. Phân tích theo phần tử lớn nhất:
    - Dựa trên các Vector văn bản trong $U_{100}\Sigma_{100}$ hoặc các Vector đặc trưng trong $\Sigma_{100}V_{100}^{\top}$, ta chọn các mẫu hoặc các đặc trưng có giá trị phần tử lớn nhất ở một chủ đề cụ thể.
    - Điều này giúp xác định các văn bản hoặc các Tokens đại diện rõ nét nhất cho chủ đề đó.
2. Phân tích theo độ tương đồng Cosine:
    - Tính toán độ tương đồng Cosine trên các Vector văn bản $U_{100}\Sigma_{100}$ hoặc các Vector đặc trưng $\Sigma_{100}V_{100}^{\top}$, tương tự như với ma trận TD W_{3500}.
    - Phương pháp này giúp đo lường mức độ liên quan giữa các văn bản và các Tokens với các chủ đề tiềm ẩn, cung cấp một góc nhìn trực quan hơn về cấu trúc của bộ Corpus.

4.4.1 Phân tích theo phần tử lớn nhất

Dưới đây là tóm tắt kết quả phân tích chủ đề tiềm ẩn theo phần tử lớn nhất cho các Tokens:

Topic	Tokens
Topic 1	life, help, die, redflag, kill, tired, pain, suicidal, hate, job
Topic 2	movie, film, dvd, story, scene, plot, funny, star, horror, actor
Topic 3	movie, film, watch, sorry, funny, acting, story, seen, plot, actor
Topic 4	movie, die, death, plot, cinema, humor, funny, mystery, watch, seen
Topic 5	reddit, discord, funny, minecraft, idk, pls, lol, meme, cuz, joke

Bảng 9: Một vài chủ đề và các Tokens đại diện cho chúng (đã được chọn lọc)

Sử dụng LSA, dữ liệu được giảm chiều xuống 5 chủ đề, bảng trên thể hiện các Tokens nổi bật được chọn lọc từ 135 token đại diện đầu tiên. Phương pháp này cung cấp thông tin giá trị, hỗ trợ hình thành giả thuyết về ý nghĩa các chủ đề, đồng thời chứng minh hiệu quả trong phát hiện và phân biệt yếu tố ngữ nghĩa, linh hoạt theo số chiều và đối tượng phân tích.

Ví dụ, chủ đề 1 liên quan rõ rệt đến lớp 1, trong khi lớp 0 gắn với phim ảnh, mạng xã hội. Đặc biệt, ở chủ đề 4, các token như "die," "death," "movie," "plot," và "character" gợi ý bối cảnh kịch bản phim thay vì tự hại. Chủ đề 5 chứa từ lóng, viết tắt và tên nền tảng trực tuyến, thể hiện sự đa dạng ngôn ngữ. Kết quả khẳng định LSA là công cụ mạnh trong khám phá mối quan hệ giữa Tokens và ngữ cảnh.

Rank	Document Excerpt (Topic 1)
1	(Class 1) yet another fucking post even know happened feel bad swear mood swing easily distinct ...
2	(Class 1) fuck made post talked situation little friend leaving college parent really like care long im fed re ...
3	(Class 1) gonna long rant feel free skip want feel fucking hopeless wrote redflag note month ago ...
4	(Class 1) actually feel alright thats past year going quite well outside ive started leaving house pretty ...
5	(Class 1) hopeless reddit im longtime lurker need help family friend connected avenue communication ...
Rank	**Document Excerpt (Topic 2)**
1	(Class 0) still cant describe feel received site rare movie little spoken known difficult find even fan...
2	(Class 0) people chase movie featuring audrey seem understand character really audrey real life ...
3	(Class 0) reason movie better known appreciated seems subject controversial subject movie never got ...
4	(Class 0) spielberg first dramatic film letdown beautifully made film without flaw life ...
5	(Class 0) film blade master may cheap clumsy appearance sometimes share thought problem way ...

Bảng 10: Các văn bản đại diện cho chủ đề 1 và chủ đề 2

Kết quả trên thu được khi xét các văn bản đại diện hàng đầu cho chủ đề 1 và chủ đề 2. Ta có thể thấy các văn bản này càng củng cố hơn cho ý nghĩa sâu xa của các chủ đề tiềm ẩn. Thậm chí các văn bản đại diện cho chủ đề 1 đều thuộc lớp 1 và điều tương tự đúng với chủ đề 2 và lớp 0.

LSA không chỉ giảm nhiễu và tối ưu hóa chiều dữ liệu cho ma trận TD W_{3500}, mà còn cung cấp hướng giải thích sâu rộng hơn so với việc xem xét từng Token riêng lẻ.

Qua những kết quả trên cho thấy LSA là một phương pháp vô cùng hiệu quả để phân tích, giảm nhiễu và giảm chiều khi làm việc với các bộ Corpus. Tuy nhiên, do bản chất vẫn sử dụng ma trận TD, LSA dựa vào không gian chủ đề tiềm ẩn rất nhiều — những chủ đề này vẫn cần sự phân tích kĩ càng để có thể phần nào giải thích được.

4.4.2 Phân tích theo độ tương đồng Cosine

Tương tự như với ma trận TD W_{3500}, ta có thể áp dụng độ tương đồng Cosine lên các hàng của $U_{100}\Sigma_{100}$ và các cột của $\Sigma_{100}V_{100}^{\top}$ để lần lượt phân tích các văn bản và các Tokens trong không gian chủ đề tiềm ẩn.

Do đặc điểm của các Vector này, khả năng tận dụng độ tương đồng Cosine cũng được trọn vẹn hơn do có thể có giá trị từ $[-1, 1]$ thay vì chỉ từ $[0, 1]$ như với các Vector của ma trận TD W_{3500}.

Dưới đây là ví dụ cho quá trình phân tích chủ đề tiềm ẩn theo độ tương đồng Cosine cho các văn bản:

Doc. Index	**Class**	S_c	**Doc. Index**	**Class**	S_c	**Doc. Index**	**Class**	S_c
15464 (Query)	0	-	1841 (Query)	1	-	23176 (Query)	1	-
15852	0	0.733	10693	1	0.685	8827	1	0.726
11483	0	0.723	9854	1	0.668	14673	0	0.718
19652	0	0.713	7124	1	0.667	17580	0	0.694
2355	0	0.695	19741	1	0.659	11006	1	0.661
19563	0	0.684	11627	1	0.646	1787	1	0.649

Bảng 11: Các văn bản có độ tương đồng Cosine lớn nhất với các văn bản được chọn (LSA)

Những kết quả trên phần lớn cho các giá trị cao hơn so với việc áp dụng độ tương đồng Cosine trực tiếp lên các hàng của ma trận TD W_{3500}, cho thấy rằng có thể trong không gian chủ đề tiềm ẩn, các văn bản được đại diện một cách trung thực hơn. Ngoài ra đa số các văn bản tương đồng nhất cũng có lớp giống với các văn bản được chọn.

Sau đây là ví dụ cho quá trình phân tích chủ đề tiềm ẩn theo độ tương đồng Cosine cho các Tokens:

Tokens	**"suicidal"**	**"depression"**	**Tokens**	**"movie"**	**"film"**
ideation	0.845	-	moviebr	0.908	-
thought	0.788	-	theater	0.814	-
intrusive	0.736	-	bruce	0.762	-
tendency	0.662	-	imdb	0.758	-
battling	0.636	-	enjoyed	0.720	-
actively	0.556	-	complaint	0.697	-
inpatient	0.517	-	collection	0.684	-
ocd	0.510	0.705	flick	0.675	-
lately	0.487	-	recommend	0.669	-
frequent	0.482	-	itbr	0.667	-
severe	-	0.861	filmbr	-	0.929
anxiety	-	0.801	cinema	-	0.902
diagnosed	-	0.782	dialogue	-	0.887
disorder	-	0.743	festival	-	0.872
symptom	-	0.690	cinematography	-	0.867
depressive	-	0.660	filmmaker	-	0.866
clinical	-	0.659	viewing	-	0.859
struggling	-	0.647	remarkable	-	0.850
manic	-	0.644	beautifully	-	0.830
-	-	-	narrative	-	0.827

Bảng 12: Các Tokens có độ tương đồng Cosine lớn nhất với các Tokens được chọn lọc (LSA)

Kết quả trên cho thấy phương pháp này hiệu quả hơn so với TF-IDF, khi các Tokens tương đồng có ý nghĩa hoặc sát nghĩa hơn. Lưu ý, các Tokens kết thúc bằng ký tự "br" là tàn dư của yếu tố HTML, chưa được loại bỏ hoàn toàn trong quá trình làm sạch Corpus. Những Tokens này thường xuất hiện cùng các Tokens được chọn, nhưng không nên hiểu rằng, ví dụ, "moviebr" tương đồng với "movie" chỉ vì chứa từ "movie". Phân tích này giúp phát hiện các vấn đề trong một bộ Corpus, từ đó cải thiện quá trình tiền xử lý nếu cần.

Ưu điểm lớn của độ tương đồng Cosine là khả năng áp dụng trên mọi văn bản và Tokens bất kỳ. Việc phân tích sâu đến đâu phụ thuộc vào mức độ đầu tư công sức. Ngoài việc tìm kiếm các giá trị tương đồng lớn nhất, ta cũng có thể khai thác thêm thông tin từ các giá trị nhỏ nhất, hoặc so sánh các đối tượng cụ thể để có những góc nhìn chi tiết hơn.

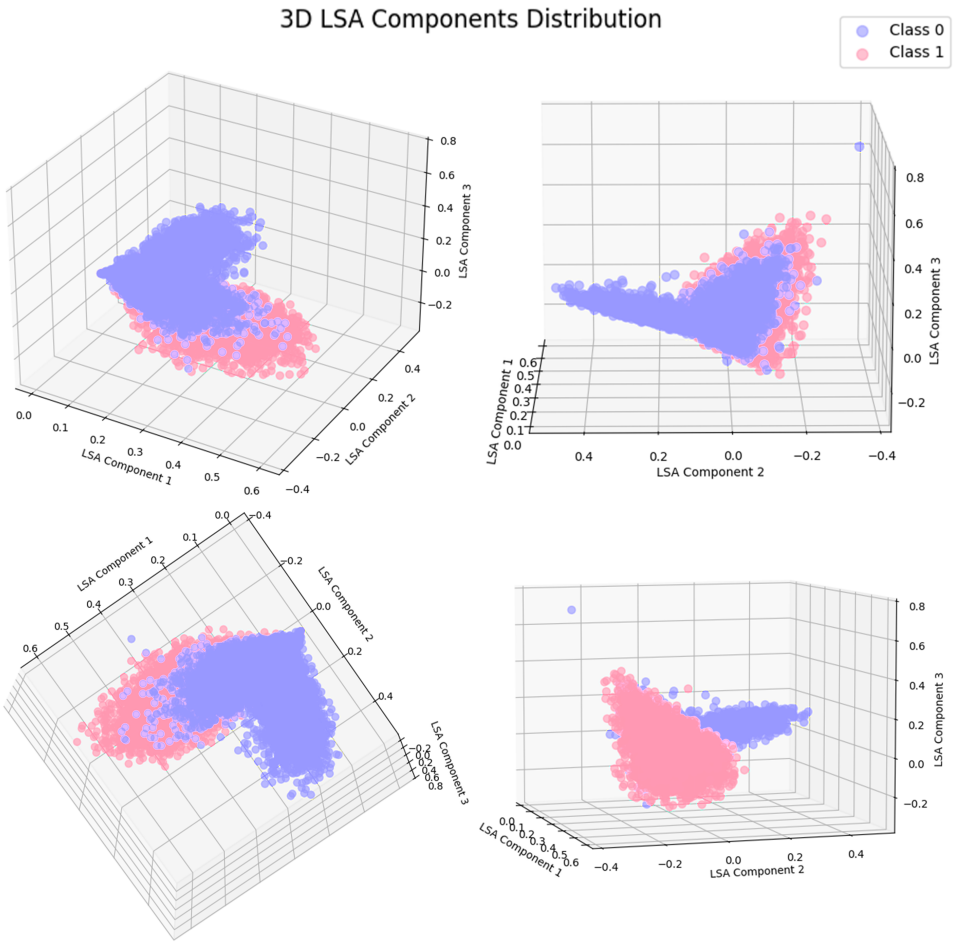
4.5 Trực quan hóa và Biplots

4.5.1 Trực quan hóa

Bằng việc áp dụng LSA lên ma trận W_{3500} và chọn $k = 2$ hoặc $k = 3$ để thu được $U_2\Sigma_2$ hoặc $U_3\Sigma_3$, ta có thể trực quan hóa các văn bản trong không gian chủ đề tiềm ẩn bằng biểu đồ Scatter Plot.

Việc giảm số chiều xuống 2 hoặc 3 tương đương với giả định rằng văn bản chỉ có 2 hoặc 3 chủ đề tiềm ẩn, dẫn đến lượng thông tin giữ lại khá hạn chế. Do đó, kết quả trực quan này chủ yếu được dùng để hình dung phân bố, thay vì đưa ra kết luận chặt chẽ.

Trước hết, ta trực quan hóa các văn bản trong không gian ba chiều tìm bởi LSA:

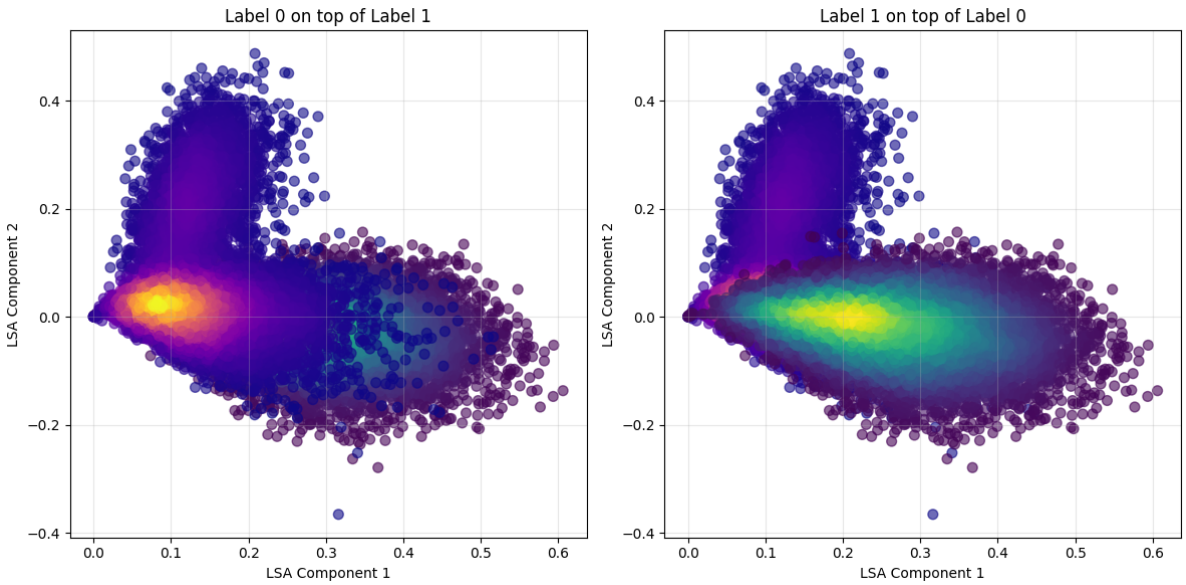


Hình 5: Các mẫu văn bản trong không gian ba chiều được tìm bởi LSA từ nhiều góc nhìn

Kết quả trực quan ban đầu có phần khó hiểu, nhưng có thể rút ra một số nhận xét:

- Hai cụm tương ứng với lớp 0 và 1 có hình dạng khá phức tạp.
- Các cụm có xu hướng "đan xen" với nhau.
- Cụm của lớp 0 có một phần nhô ra, không giao với cụm của lớp 1.

Dựa trên các quan sát này, ta sử dụng thang màu để hiển thị mật độ: điểm càng dày, màu càng sáng. Từ đó, tạo hai biểu đồ Scatter Plot trong không gian hai chiều tìm bởi LSA, với cách hiển thị như sau: lớp 0 nằm trên lớp 1 (bên trái) và lớp 1 nằm trên lớp 0 (bên phải).



Hình 6: Các mẫu văn bản trong không gian hai chiều được tìm bởi LSA

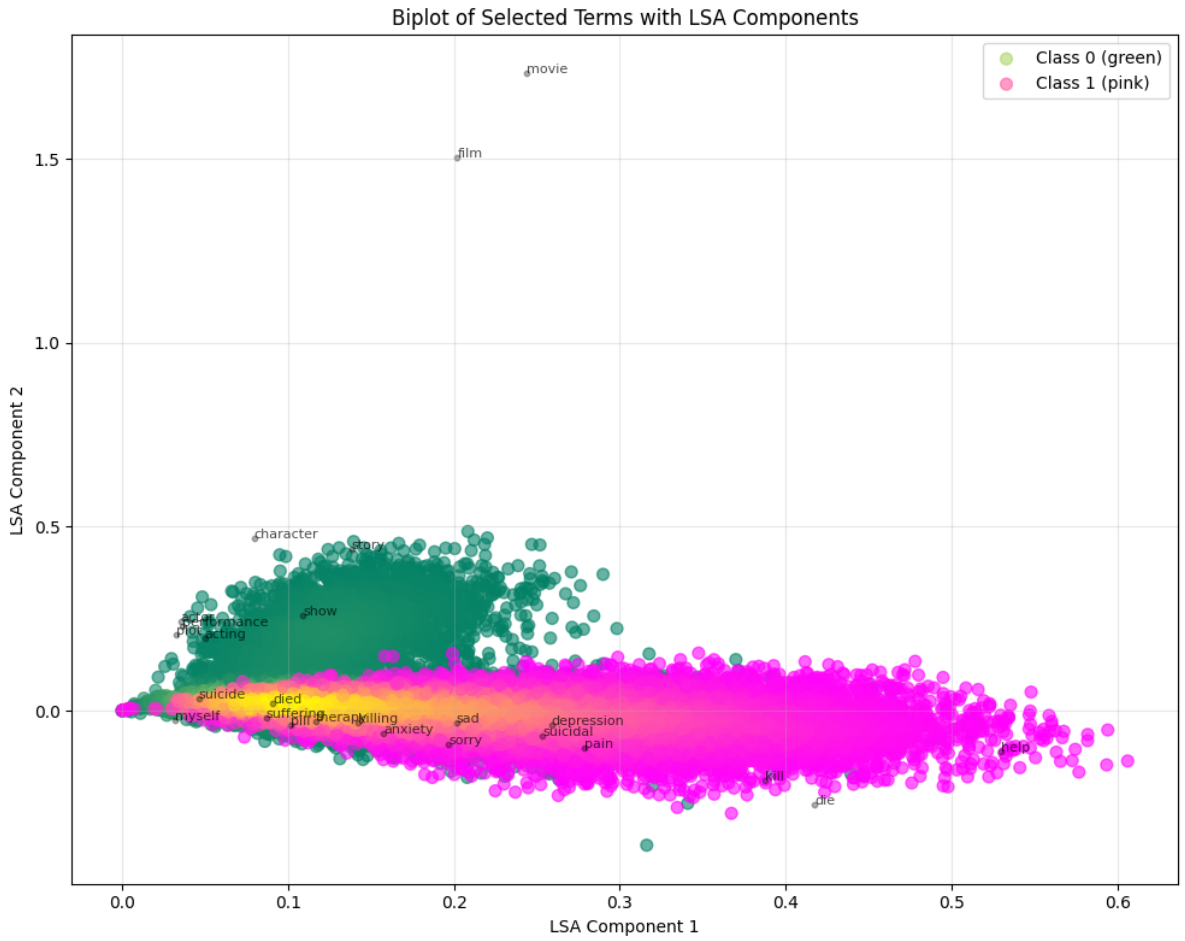
Kết quả trên cho thấy rõ hơn đặc điểm giao thoa giữa hai lớp và phần nhô ra đặc trưng của lớp 0. Phần sáng nhất trên biểu đồ phản ánh mật độ cao nhất, tức khu vực tập trung nhiều văn bản nhất. Các cụm vẫn giữ cấu trúc dày đặc ở tâm và thưa dần về phía rìa, đặc biệt rõ rệt ở lớp 1.

Những phát hiện này củng cố nhận định rằng phần lớn văn bản không có chủ đề rõ ràng khi phân tích sơ bộ. Điều này giải thích sự đan xen giữa hai lớp, gợi ý rằng chúng chia sẻ nhiều Tokens chung. Hiện tượng này dễ hiểu đối với dữ liệu từ mạng xã hội, vốn khác biệt so với các văn bản chính thống có cấu trúc chủ đề rõ ràng. Đáng chú ý, phần nhô ra của lớp 0 là một đặc điểm thú vị, xứng đáng được phân tích sâu hơn trong các bước tiếp theo.

4.5.2 Biplots

Ngoài việc chỉ trực quan hóa các văn bản, ta có thể sử dụng *biểu đồ Biplot* để đồng thời trực quan cả các văn bản và các đặc trưng, cho phép ta quan sát cấu trúc các văn bản trong không gian chủ đề tiềm ẩn và phân tích mối quan hệ của chúng với các Tokens.

Với $k = 2$, ta kết hợp các hàng của $U_2\Sigma_2$ và các cột của $\Sigma_2 V_2^\top$ để biểu diễn trên cùng một biểu đồ. Các Tokens đã được chọn lọc kỹ lưỡng để đảm bảo hiển thị những kết quả nổi bật nhất:



Hình 7: Biplots chứa các mẫu văn bản và các Vector Tokens trong không gian hai chiều được tìm bởi LSA

Quan sát biểu đồ Biplot, ta nhận thấy các token liên quan đến chủ đề sức khỏe tâm lý chủ yếu nằm trong vùng giao thoa giữa lớp 0 và lớp 1. Điều này cho thấy sự mơ hồ về chủ đề nội dung giữa hai lớp của các văn bản trong khu vực này.

Thú vị hơn, phần nhô ra của lớp 0 nằm cùng hướng với các Vector Tokens liên quan đến phim ảnh và điện ảnh. Điều này gợi ý rằng phần nhô ra có thể đại diện cho một chủ đề riêng biệt về phim ảnh. Ta có thể phỏng đoán rằng các văn bản thuộc khu vực này có chủ đề rõ ràng và riêng biệt về phim ảnh và điện ảnh, thể hiện qua sự rời rạc so với các văn bản còn lại.

Chương 5: Phân cụm dữ liệu

5.1 Khái quát quá trình phân cụm

Phân cụm văn bản là một công việc phổ biến trong quá trình phân tích các bộ Corpus, cho phép nhóm các văn bản có sự tương đồng về nội dung với nhau. Trong báo cáo này, ta sẽ xét 2 mô hình phân cụm đó là *K-Means* và *Gaussian Mixture Model*.

Khi sử dụng bộ dữ liệu TF-IDF, các Vector văn bản được chuẩn hóa và không chứa phần tử âm, điều này đồng nghĩa với việc chúng phân bố trong một góc dương cố định trên một *Hypersphere (siêu cầu)* với bán kính bằng 1. Khi sử dụng bộ dữ liệu LSA, các Vector văn bản trong không gian chủ đề tiềm ẩn có thể chứa phần tử âm và không bị chuẩn hóa, do đó cung cấp sự phân bố tự do hơn, tránh hiện tượng các điểm dữ liệu bị tập trung quá mức trong một góc cụ thể. Vì lý do này, ta sẽ ưu tiên sử dụng bộ dữ liệu LSA cho các mô hình phân cụm.

Chúng ta sẽ áp dụng các thuật toán phân cụm lên bộ dữ liệu LSA $U_{100}\Sigma_{100}$, sau đó gán nhãn cụm và trực quan hóa phân bổ các cụm trong không gian hai chiều tìm được bởi LSA. Cụ thể ta sẽ quan sát kết quả với 2 và 3 cụm.

5.2 Phương pháp phân cụm K-Means

5.2.1 Khái quát phương pháp

K-Means là một thuật toán phân cụm không giám sát, được sử dụng để chia bộ dữ liệu thành k cụm khác nhau dựa trên khoảng cách giữa các điểm trong không gian. Đây là một trong những thuật toán phân cụm cơ bản và phổ biến nhất.

Nguyên tắc hoạt động của K-Means bao gồm 4 bước chính:

- Khởi tạo tâm cụm
- Gán nhãn cụm
- Cập nhật tâm cụm
- Lặp lại đến hội tụ

Thuật toán tiếp tục tối ưu hóa vị trí của các tâm cụm nhằm giảm thiểu tổng bình phương khoảng cách giữa các điểm dữ liệu và tâm cụm tương ứng.

5.2.2 Hàm tổn thất

Cho bộ dữ liệu $X = \{x_1, x_2, \ldots, x_N\}$ và $x_i \in \mathbb{R}^n$, hàm tổn thất của thuật toán K-Means là:[30,36]

$$J = \sum_{i=1}^{K} \sum_{x \in C_i} \|x - \mu_i\|^2$$

Trong đó:

- K là tổng số cụm.
- $x \in X$ là một điểm dữ liệu thuộc X.
- C_i là tập chứa các điểm dữ liệu của cụm thứ i.
- μ_i là tâm của cụm thứ i.

5.2.3 Các bước thuật toán

Bước 1 - Khởi tạo tâm cụm: Khởi tạo k tâm cụm $\{\mu_1, \mu_2, \ldots, \mu_k\}$ ngẫu nhiên.

Bước 2 - Gán nhãn cụm: Gán các điểm dữ liệu với có khoảng cách gần nhất với tâm cụm μ_i với nhãn C_i:

$$C_i = \{x_j : \|x_j - \mu_i\|^2 \leq \|x_j - \mu_l\|^2, \forall l = 1, 2, \ldots, k\}$$

Bước 3 - Cập nhật tâm cụm: Dịch chuyển tâm cụm cho phù hợp với các điểm dữ liệu được gán:

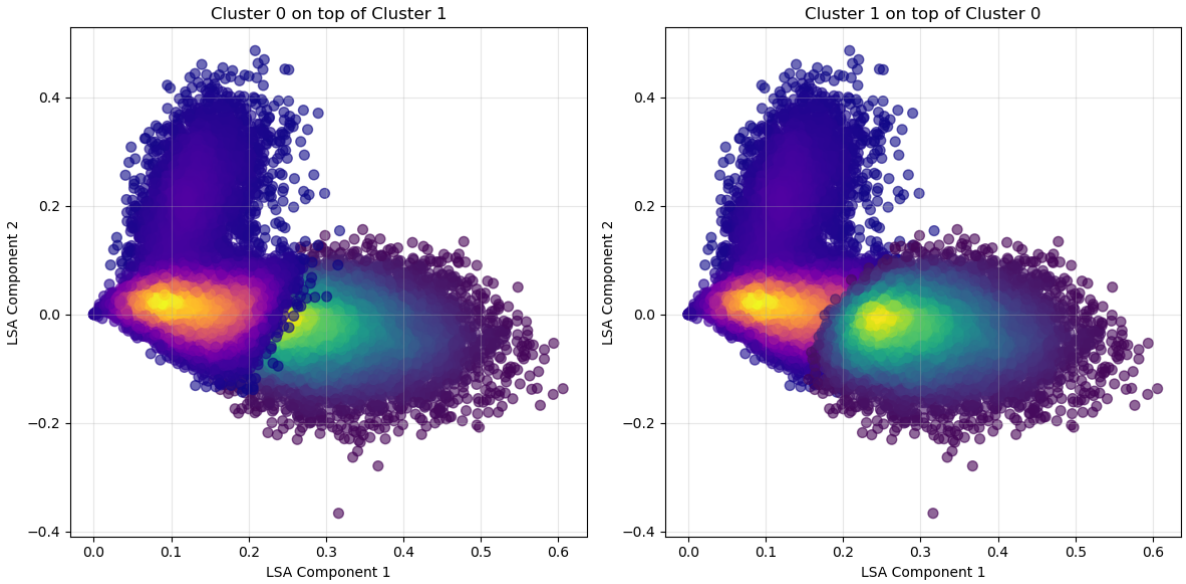
$$\mu_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$$

Bước 4 - Kiểm tra hội tụ: Lặp lại **Bước 2** và **Bước 3** cho tới khi thuật toán hội tụ:

$$|\Delta J| = |J^{\text{new}} - J| < \epsilon$$

5.2.4 Trực quan kết quả phân cụm K-Means

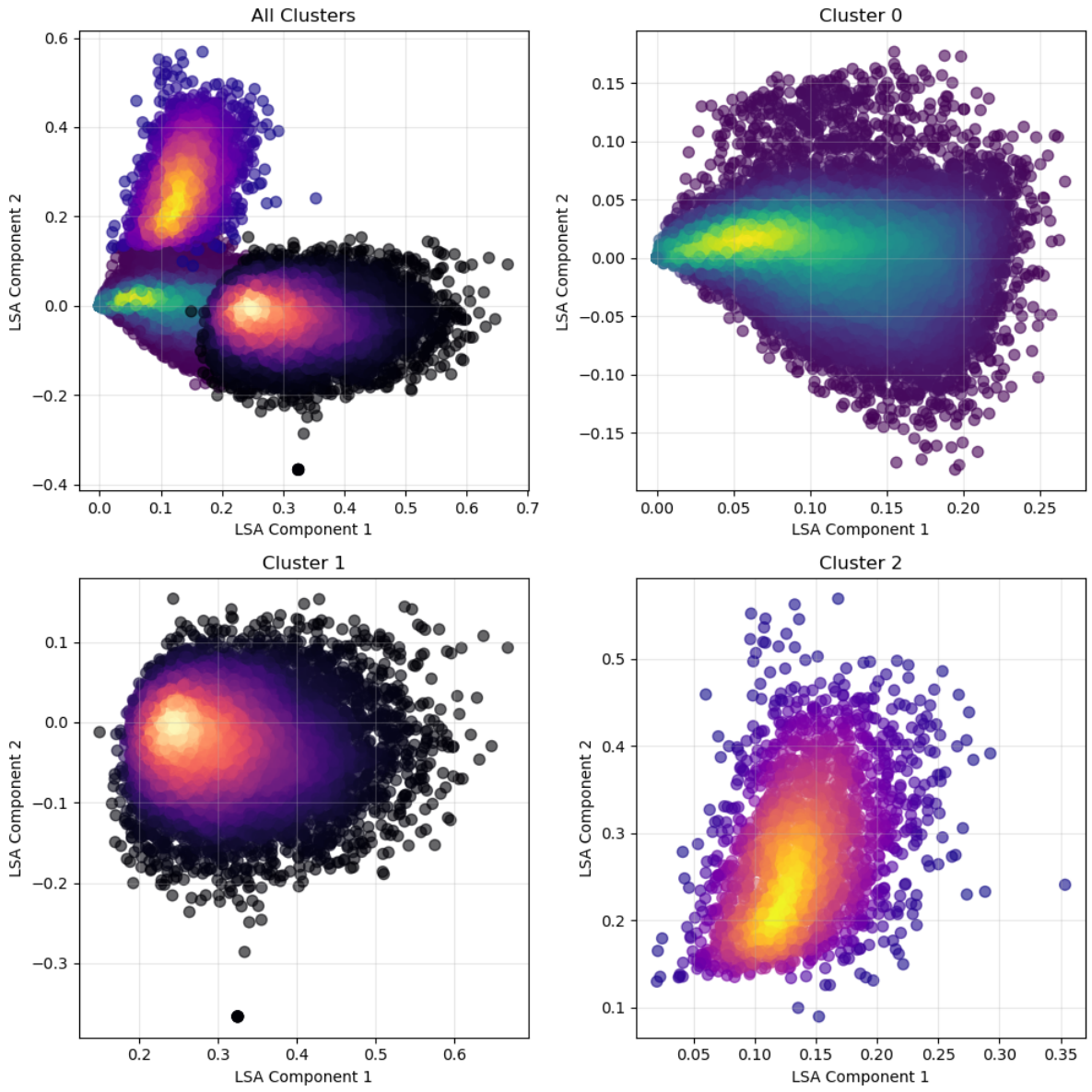
Chúng ta áp dụng thuật toán K-Means để phân cụm trên bộ dữ liệu LSA với $k = 2$, thu nhãn cụm và trực quan phân bổ của các cụm với LSA trong không gian hai chiều:



Hình 8: Kết quả phân cụm K-Means với hai cụm

Quan sát kết quả, ta nhận thấy rằng ngay cả trong không gian nhiều chiều, K-Means vẫn có khả năng phát hiện các cấu trúc cụm tương đối rõ ràng và đồng nhất. Tuy nhiên, một nhược điểm đáng chú ý của K-Means là không thể xử lý hiệu quả các cụm có sự chồng lấn. Đây là một hạn chế đặc trưng của K-Means, đặc biệt khi dữ liệu có cấu trúc phức tạp, không dễ phân tách, đan xen vào nhau.

Sau đây chúng ta thực hiện phân cụm K-Means lên bộ dữ liệu LSA với $k = 3$, thu nhãn cụm và trực quan phân bổ của các cụm với LSA trong không gian hai chiều:



Hình 9: Kết quả phân cụm K-Means với ba cụm

Khi chạy K-Means với $k = 3$, kết quả cho thấy thuật toán đã phân tách thành công phần nhô ra của cụm lớp 0, mà được giả định là đại diện cho chủ đề phim ảnh, điện ảnh. Kết quả này gợi ý rằng việc tăng số lượng cụm trong K-Means có thể mang lại những khám phá sâu hơn về chủ đề nội dung của các văn bản trong bộ Corpus.

5.3 Phương pháp phân cụm Gaussian Mixture Model

5.3.1 Khái quát phương pháp

Gaussian Mixture Model (GMM) là một thuật toán phân cụm không giám sát, dựa trên giả thuyết rằng dữ liệu được tạo thành từ một hỗn hợp các phân phối Gaussian (phân phối chuẩn). Điểm khác biệt chính so với K-Means là GMM có khả năng xử lý các cụm "chồng lấn" và cung cấp phân cụm "mềm" (soft clustering), biểu diễn xác suất một điểm thuộc về từng cụm thay vì chỉ gán một nhãn cụm "cứng" (hard clustering).

Thuật toán phổ biến nhất được sử dụng cho các mô hình GMM là *Estimation-Maximization (EM)*, dựa trên lý thuyết xác suất thống kê nhằm tối ưu hóa các tham số của mô hình hỗn hợp Gaussian. Mặc dù tính toán của GMM phức tạp hơn và thời gian thực thi lâu hơn so với K-Means, sự đánh đổi này mang lại tính linh hoạt cao hơn và khả năng phân cụm hiệu quả trong các tình huống dữ liệu phức tạp.

5.3.2 Tổng quan mô hình và hàm mục tiêu

Mô hình GMM được đại diện bởi các phân bố chuẩn đa chiều trộn. Cho bộ dữ liệu $X = \{x_1, x_2, \ldots, x_N\}$ và $x_i \in \mathbb{R}^n$, hàm hợp lý của 1 điểm dữ liệu x_i trong mô hình là:[32]

$$p(x_i) = \sum_{k=1}^{K} \pi_k g(x_i \mid \mu_k, \Sigma_k)$$

Trong đó K là tổng số cụm và $\pi_k \geq 0$ được gọi là *hệ số trộn*, thỏa mãn $\sum_{k=1}^{K} \pi_k = 1$. $g(x_i \mid \mu_k, \Sigma_k)$ là phân bố chuẩn đa chiều thứ k trong mô hình:

$$g(x_i \mid \mu_k, \Sigma_k) = \frac{1}{(2\pi)^{n/2}|\Sigma|^{1/2}} \exp(-\frac{1}{2}(x_i - \mu_k)^\top \Sigma^{-1}(x_i - \mu_k))$$

Ta định nghĩa các tham số của mô hình là $\theta = \{\mu_k, \Sigma_k\}_{k=1}^{K}$ và $\Theta = \{\pi_k, \theta_k\}_{k=1}^{K}$. Từ phương trình *hợp lí cực đại*, ta suy ra *hàm Logarit hợp lý cực đại* để đơn giản hóa tính toán:[28]

$$L(\Theta; X) = \prod_{i=1}^{N} p(x_i) = \prod_{i=1}^{N} \sum_{k=1}^{K} \pi_k g(x_i \mid \theta_k) \quad ; \quad \log L(\Theta; X) = \sum_{i=1}^{N} \log \left(\sum_{k=1}^{K} \pi_k g(x_i \mid \theta_k) \right)$$

Lúc này, hàm mục tiêu của mô hình được định nghĩa là:

$$\hat{\Theta} = \arg\max_{\Theta} \log L(\Theta; X)$$

5.3.3 Thuật toán Estimation-Maximization

Do việc tính trực tiếp các tham số của mô hình là bất khả thi, thuật toán *Estimation-Maximization (EM)* hay còn gọi là *Cực đại hóa Kỳ vọng* được sử dụng để liên tục ước tính tham số Θ. Thuật toán EM tối ưu $\log L\ (\Theta; X)$ bằng cách tận dụng khái niệm *biến ẩn*.

Ta định nghĩa tập biến ẩn $Z = \{Z_1, \ldots, Z_k\}$, $Z_k = \{z_{1k}, z_{2k}, \ldots, z_{Nk}\}$ và $z_{ik} \in \{0, 1\}$ đại diện cho nhãn cụm. Khi xét cả biến ẩn Z, hàm Logarit hợp lý cực đại hoàn chỉnh trở thành:[9,33]

$$\log L(\Theta; \{X; Z\}) = \sum_{i=1}^{N} \sum_{k=1}^{K} \gamma_{ik} \log \left(\pi_k g(x_i \mid \theta_k) \right)$$

Trong đó xác xuất tiên nghiệm $\gamma_{ik} = P(z_{ik} = 1 \mid x_i)$ được tính trực tiếp trong **Bước E** và các tham số được cập nhật trong **Bước M** của thuật toán EM. Sự hội tụ của thuật toán EM được đảm bảo bởi *bất đẳng thức Jensen*.[28]

Bằng việc tính đạo hàm từng phần của $\log L(\Theta; \{X; Z\})$ với từng tham số, ta thu được các công thức tính tham số ở **Bước M**.

Các bước thuật toán:[32]

Bước 1 - Khởi tạo tham số: Khởi tạo tham số $\Theta = \{\pi_k, \theta_k\}_{k=1}^{K}$ ngẫu nhiên.

Bước 2 - Bước E (Estimation): Tính xác xuất tiên nghiệm dựa trên định lý Bayes:

$$\gamma_{ik} = \frac{\pi_k g(x_i \mid \theta_k)}{\sum_{j=1}^{K} \pi_j g(x_i \mid \theta_j)}$$

Bước 3 - Bước M (Maximization): Cập nhật các tham số:

$$\pi_k = \frac{\sum_{i=1}^{N} \gamma_{ik}}{N} \quad ; \quad \mu_k = \frac{\sum_{i=1}^{N} \gamma_{ik} x_i}{\sum_{i=1}^{N} \gamma_{ik}} \quad ;$$

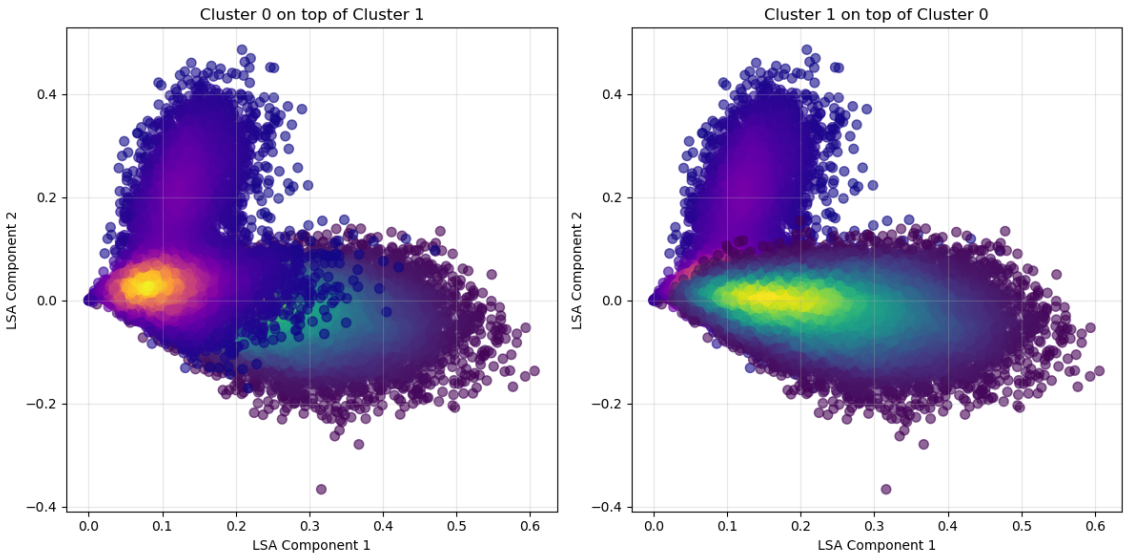
$$\Sigma_k = \frac{\sum_{i=1}^{N} \gamma_{ik}(x_i - \mu_k)(x_i - \mu_k)^\top}{\sum_{i=1}^{N} \gamma_{ik}}$$

Bước 4 - Kiểm tra hội tụ: Lặp lại **Bước 2** và **Bước 3** cho tới khi thuật toán hội tụ:

$$|\Delta L| = |L(\Theta^{\text{new}}; \{X; Z\}) - L(\Theta; \{X; Z\})| < \epsilon$$

5.3.4 Trực quan kết quả phân cụm GMM

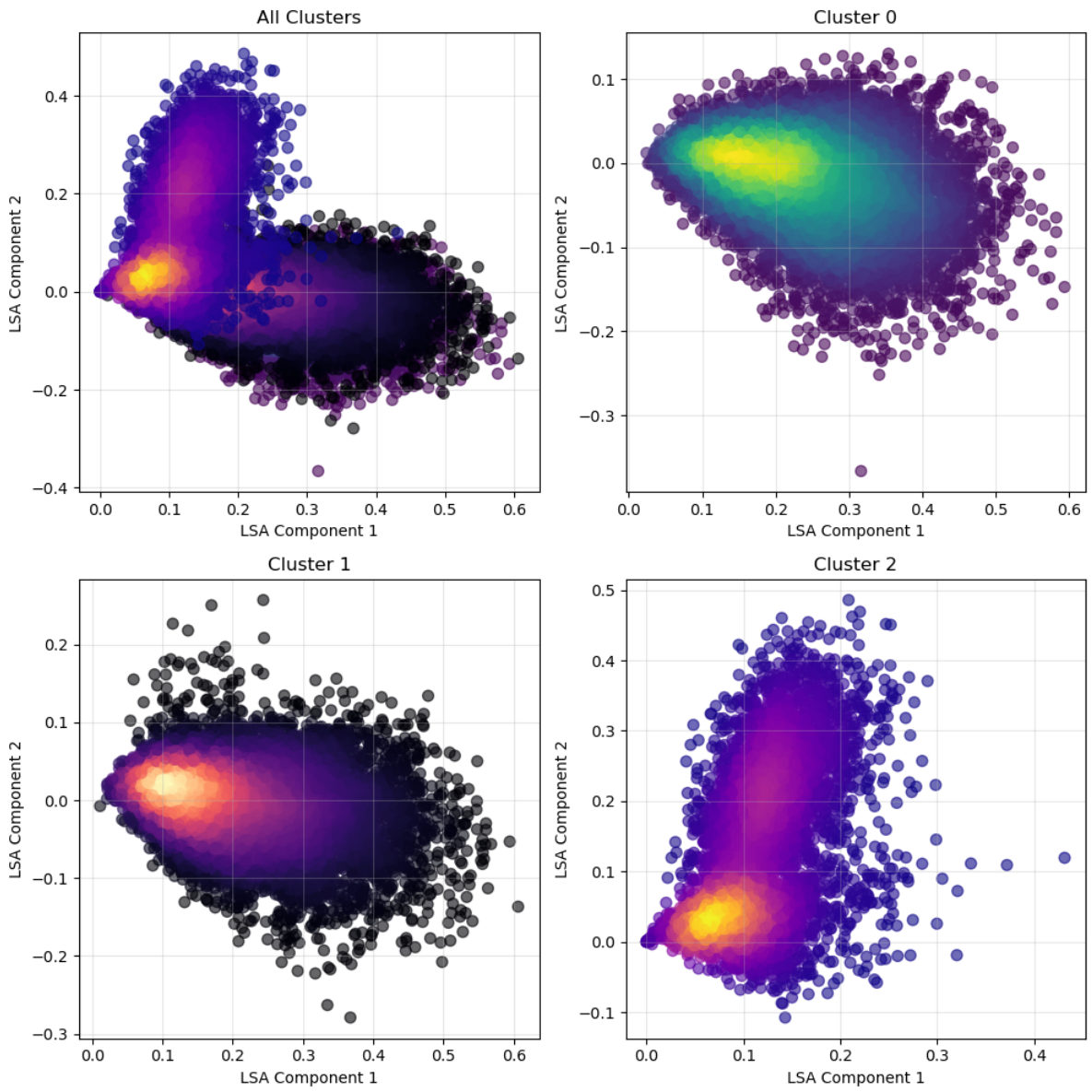
Sau đây chúng ta thực hiện áp dụng phận cụm GMM lên bộ dữ liệu LSA với số cụm là 2, thu nhãn cụm và trực quan phân bổ của các cụm với LSA trong không gian hai chiều:



Hình 10: Kết quả phân cụm GMM với hai cụm

Ta thấy rõ được lợi thế của GMM so với K-Means khi mà phân bổ của hai cụm vô cùng tương đồng với phân bố lớp gốc, cho thấy khả năng xử lý các cụm đan vào nhau của GMM. Đặc biệt cụm 1 có hình dáng gần như giống hoàn toàn với phân bổ gốc của lớp 1.

Sau đây chúng ta thực hiện áp dụng phận cụm GMM lên bộ dữ liệu LSA với số cụm là 3, thu nhãn cụm và trực quan phân bổ của các cụm với LSA trong không gian hai chiều:



Hình 11: Kết quả phân cụm GMM với ba cụm

Kết quả của GMM khi chạy với 3 cụm cho thấy khả năng phân cụm văn bản theo chủ đề hiệu quả tương tự, tất nhiên với các lợi thế đã được đề cập của GMM khi chạy với 2 cụm.

Khái quát quá trình phân loại

Bộ dữ liệu được sử dụng

Bộ dữ liệu gốc có 27977 mẫu, sau khi làm sạch ta có bộ dữ liệu gồm 23240 mẫu với phân bổ lớp là 10592 mẫu thuộc lớp 0 ($\sim$45.6%) và 12648 mẫu thuộc lớp 1 ($\sim$54.4%). Như vậy, đây là một bài toán phân loại nhị phân (gồm 2 lớp 0 và 1).

Trong suốt quá trình phân loại, ta sẽ sử dụng:

- $W_{3500} \in \mathbb{R}^{23240\times3500}$ là bộ dữ liệu trích xuất bởi TF-IDF. Mỗi hàng là một Vector $D^{(\text{TF-IDF})} \in \mathbb{R}^{3500}$ đại diện cho các văn bản được Vector hóa. Mỗi cột đại diện cho một đặc trưng, gồm các trọng số TF-IDF hay có thể hiểu là độ quan trọng của 1 Token đối với từng mẫu văn bản. Bộ dữ liệu TF-IDF được chuẩn hóa theo hàng (theo mẫu).
- $U_{100}\Sigma_{100} \in \mathbb{R}^{23240\times100}$ là bộ dữ liệu giảm chiều bởi LSA áp dụng lên ma trận TD W_{3500}. Mỗi hàng là một Vector $D^{(\text{LSA})} \in \mathbb{R}^{100}$ đại diện cho các văn bản được biểu diễn trong không gian chủ đề tiềm ẩn. Mỗi cột đại diện cho một đặc trưng, gồm các trọng số thể hiện độ quan trọng của một chủ đề tiềm ẩn với từng mẫu văn bản. Bộ dữ liệu LSA được chuẩn hóa theo cột (theo đặc trưng).

Ta sẽ sử dụng *tập Train-Test Split* với tỉ lệ 80% Train (18592 mẫu) và 20% Test (4648 mẫu) với hạt giống ngẫu nhiên 42, sau đây là thông số về phân bổ lớp:

Label	Count
1	10099
0	8493

Bảng 13: Phân bổ lớp trong tập Train

Label	Count
1	2549
0	2099

Bảng 14: Phân bổ lớp trong tập Test

Thang đo đánh giá phân loại

Ta sẽ huấn luyện các mô hình bằng tập Train (tập huấn luyện), sau đó thử nghiệm bằng tập Test (tập kiểm thử), đánh giá trên các thang đo:

- Train Accuracy (Độ chính xác trên tập Train).
- Test Accuracy (Độ chính xác trên tập Test).
- Precision (Độ tập trung).
- Recall (Độ bao phủ).
- F1 Score (Điểm F1).

Ta xét F1 Score làm thang đo chính để tìm ra các mô hình tốt nhất. Ngoài ra, ta cũng sẽ đo các thông số thời gian nổi bật như Fit Time (s) hoặc Test Time (s).

Chú ý: Các thông số thời gian sẽ được bỏ qua nếu không nổi bật (hầu hết là do vô cùng thấp).

Chương 6: Phân loại bằng mô hình K-Nearest Neighbors

6.1 Nội dung phương pháp

6.1.1 Khái quát phương pháp

K-Nearest Neighbors (K-NN) là một trong những mô hình đơn giản nhất trong nhóm mô hình học máy giám sát. K-NN hoạt động bằng cách ghi nhớ toàn bộ tập huấn luyện, sau đó với mỗi điểm cần xét, K-NN sẽ tính toán theo một hàm khoảng cách giữa điểm đó tới toàn bộ tập huấn luyện và đưa ra dự đoán bằng K *"láng giềng gần nhất"*. K-NN có khả năng xử lý cả các bài toán phân loại và hồi quy, trong phần này ta sẽ xây dựng một mô hình K-NN phân loại sử dụng cả hàm khoảng cách Euclidean và Cosine.

K-NN là một mô hình vô cùng đơn giản, ít tham số và dễ giải thích. Tuy nhiên hiệu suất có thể bị ảnh hưởng do dữ liệu với số chiều rất cao và nhiễu. Ngoài ra với các bộ dữ liệu lớn, thời gian kiểm thử có thể không được tối ưu.

6.1.2 Hàm khoảng cách - Euclidean và Cosine

Cho bộ dữ liệu N mẫu, n đặc trưng là $X = \{(x_i, y_i)\}_{i=1}^{N}$ với $x_i \in \mathbb{R}^n$ và có C lớp sao cho $y_i \in \{1, 2, \ldots, C\}$. Gọi điểm dữ liệu cần xét là x_q, ta định nghĩa các hàm khoảng cách:

Hàm khoảng cách Euclidean:

$$d(x_q, x_i) = \sqrt{\sum_{j=1}^{n} (x_{qj} - x_{ij})^2}$$

Độ tương đồng Cosine:

$$S_c(x_q, x_i) = \frac{x_q \cdot x_i}{\|x_q\| \cdot \|x_i\|}$$

6.1.3 Cách mô hình đưa ra dự đoán

Với k là số "hàng xóm" cần xét, ta tính các tập hàng xóm dựa trên hàm khoảng cách:

$$N_{L2}^{(k)} = \{(x_i, y_i) \mid d(x_q, x_i) = \min_{j=1,\ldots,k} d(x_q, x_i)\}$$

Hoặc:

$$N_{Cos}^{(k)} = \{(x_i, y_i) \mid S_c(x_q, x_i) = \max_{j=1,\ldots,k} S_c(x_q, x_i)\}$$

Khi đó, mô hình sẽ đưa ra dự đoán về lớp y_q của điểm cần xét x_q bằng cách xét biểu quyết đa số của nhãn của các k điểm gần nhất trong tập hàng xóm:

$$y_q = \text{mode}(\{y_i \mid (x_i, y_i) \in N_{L2}^{(k)}(x_q)\})$$

Hoặc:

$$y_q = \text{mode}(\{y_i \mid (x_i, y_i) \in N_{Cos}^{(k)}(x_q)\})$$

6.2 Áp dụng mô hình và kết quả

Sau đây, chúng ta áp dụng mô hình K-NN với hàm khoảng cách Euclidean và Cosine với các giá trị k khác nhau cho cả bộ dữ liệu TF-IDF và bộ dữ liệu LSA:

6.2.1 Chạy trên bộ dữ liệu TF-IDF

	Train Accuracy		Test Accuracy		F1 Score		Precision		Recall		Test Time (s)	
Distance	cosine	euclidean	cosine	euclidean	cosine	euclidean	cosine	euclidean	cosine	euclidean	cosine	euclidean
K												
1	1.000	1.000	0.801	0.459	0.801	0.304	0.806	0.611	0.801	0.459	2.193	2.282
5	0.901	0.807	0.869	0.749	0.869	0.743	0.869	0.807	0.869	0.749	2.767	2.854
10	0.892	0.869	0.873	0.843	0.873	0.843	0.873	0.858	0.873	0.843	2.740	2.818
50	0.884	0.889	0.883	0.888	0.881	0.887	0.889	0.892	0.883	0.888	2.723	2.867
100	0.865	0.870	0.864	0.868	0.861	0.866	0.880	0.882	0.864	0.868	2.697	2.828
200	0.830	0.833	0.831	0.836	0.824	0.829	0.863	0.866	0.831	0.836	2.708	2.862
500	0.764	0.765	0.769	0.771	0.750	0.752	0.833	0.834	0.769	0.771	2.805	2.960
1000	0.707	0.708	0.708	0.709	0.670	0.670	0.807	0.808	0.708	0.709	3.054	3.730
2000	0.656	0.656	0.661	0.661	0.599	0.600	0.790	0.790	0.661	0.661	3.441	4.781
4000	0.597	0.597	0.604	0.604	0.502	0.502	0.770	0.770	0.604	0.604	4.233	4.159

Hình 12: K-NN sử dụng Euclidean và Cosine với các giá trị k khác nhau trên bộ dữ liệu TF-IDF

6.2.2 Chạy trên bộ dữ liệu LSA

	Train Accuracy		Test Accuracy		F1 Score		Precision		Recall		Test Time (s)	
Distance	cosine	euclidean	cosine	euclidean	cosine	euclidean	cosine	euclidean	cosine	euclidean	cosine	euclidean
K												
1	1.000	1.000	0.824	0.764	0.824	0.762	0.824	0.792	0.824	0.764	0.571	0.359
5	0.900	0.846	0.859	0.790	0.859	0.788	0.860	0.826	0.859	0.790	1.106	0.300
10	0.891	0.800	0.866	0.763	0.865	0.758	0.865	0.822	0.866	0.763	1.191	0.295
50	0.865	0.768	0.860	0.756	0.858	0.749	0.864	0.822	0.860	0.756	1.095	0.337
100	0.856	0.749	0.851	0.742	0.848	0.734	0.860	0.817	0.851	0.742	1.143	0.374
200	0.841	0.726	0.839	0.720	0.834	0.708	0.856	0.809	0.839	0.720	1.157	0.473
500	0.814	0.689	0.813	0.686	0.804	0.667	0.848	0.800	0.813	0.686	1.227	0.808
1000	0.778	0.654	0.780	0.650	0.764	0.619	0.837	0.788	0.780	0.650	1.386	1.177
2000	0.735	0.625	0.738	0.617	0.710	0.575	0.818	0.779	0.738	0.617	1.677	1.571
4000	0.688	0.606	0.690	0.603	0.644	0.554	0.799	0.779	0.690	0.603	2.462	2.218

Hình 13: K-NN sử dụng Euclidean và Cosine với các giá trị k khác nhau trên bộ dữ liệu LSA

6.3 Đánh giá mô hình

6.3.1 Nhận xét kết quả

Có thể thấy rằng các mô hình K-NN đạt hiệu suất khá tốt, đặc biệt khi ta chọn k một cách hợp lí. Sự khác biệt giữa các mô hình sử dụng bộ dữ liệu TF-IDF so với các mô hình sử dụng bộ dữ liệu LSA thấy được rõ nhất ở Test Time và ở trường hợp $k = 1$. Việc sử dụng Cosine giúp cho các mô hình cho ra kết quả cao hơn, nhanh hơn, và ổn định hơn so với sử dụng Euclidean.

Với các giá trị k thấp, hiện tượng *Overfitting (quá khớp)* xảy ra khi mà Train Accuracy vô cùng cao, đặc biệt đạt 100% khi $k = 1$ trong khi các thước đo khác khá thấp. Thú vị rằng với mô hình sử dụng bộ dữ liệu LSA, kể cả với $k = 1$ các thước đo khác vẫn khá cao so với khi sử dụng bộ dữ liệu TF-IDF.

Với các giá trị k cao, ta thấy hiện tượng *Underfitting (thiếu khớp)* xảy ra khi mà mô hình đã không thể thu nhận được cấu trúc phức tạp của bộ dữ liệu khiến cho hầu hết các thang đo đều thấp. Ngoài ra với k càng lớn, thời gian tính toán càng lâu.

6.3.2 Mô hình tốt nhất

Model	Train Acc.	Test Acc.	F1 Score	Precision	Recall	Test Time (s)
Euclidean K-NN K=50 (TF-IDF)	0.889	0.888	0.887	0.892	0.888	2.867
Cosine K-NN K=10 (LSA)	0.891	0.866	0.865	0.866	0.866	1.191

Bảng 15: Các mô hình K-NN tốt nhất theo điểm F1

6.3.3 Giải thích mô hình

Các mô hình K-NN vô cùng dễ giải thích, chúng chỉ đơn giản tìm các điểm dữ liệu gần nhất theo một hàm khoảng cách đối với điểm cần xét để đưa ra dự đoán. Ta có thể hiểu rằng với k thấp, các mô hình thường sẽ thu nhận nhiều nhiễu hơn với các đường biên phức tạp hơn trong khi với k cao, các mô hình thường sẽ có các đường biên mượt hơn nhưng cũng có thể phân loại sai khá nhiều các điểm huấn luyện. Do vậy, việc lựa chọn k phù hợp là vô cùng quan trọng cho các mô hình K-NN.

Với các mô hình K-NN của chúng ta, có thể thấy rằng các tập huấn luyện đã có thể đại diện cho cấu trúc của bộ Corpus một cách khá hiệu quả, cho ra hiệu suất cao. Ngoài ra, ta thấy đối với bộ dữ liệu LSA, có vẻ các mẫu văn bản cũng được đại diện một cách khá trung thực khi mà hiệu suất các mô hình vẫn cao dù cho bộ dữ liệu chỉ còn lại 100 chiều.

Hơn nữa, việc phân tích tập hàng xóm của các điểm dữ liệu (tương tự như phân tích độ tương đồng Cosine đối với mô hình K-NN Cosine) cũng giúp ta hiểu rõ hơn về cấu trúc của bộ dữ liệu và cách mà mô hình đưa ra dự đoán.

Chương 7: Phân loại bằng mô hình Random Forest

7.1 Nội dung phương pháp

7.1.1 Khái quát phương pháp

Random Forest (viết tắt là RF) là một mô hình học máy giám sát mạnh mẽ, có thể áp dụng cho cả bài toán phân loại và hồi quy. RF sử dụng tập hợp nhiều *cây quyết định (Decision Tree)*, kết hợp kết quả của chúng để cải thiện độ chính xác và giảm thiểu hiện tượng Overfitting, vốn thường xảy ra với một cây quyết định đơn lẻ.

Mặc dù RF thường đạt hiệu suất cao hơn một cây quyết định, việc kết hợp nhiều cây dẫn đến sự đánh đổi giữa hiệu suất và khả năng giải thích đơn giản của chúng.

Phần tiếp theo sẽ tập trung xây dựng một mô hình *Bagging Random Forest* cho bài toán phân loại.

7.1.2 Tổng quan mô hình

Một mô hình RF được mô hình hóa là 1 tập hợp các cây quyết định:

$$\mathcal{T}_{RF} = \{T_1, T_2, \dots, T_K\}$$

7.1.3 Cây quyết định

Cây quyết định về bản chất vẫn là một mô hình học máy độc lập, do vậy RF là một *mô hình Ensemble* (Một tập hợp các mô hình cùng loại). Cây quyết định hoạt động bằng cách chia bộ dữ liệu thành các phần nhỏ hơn thông qua hệ thống các nút (Node) phân tầng:

- Root Node: Nút gốc, nơi bắt đầu phân chia bộ dữ liệu.
- Internal Node: Các nút trung gian, thực hiện việc phân chia dữ liệu dựa trên một quy tắc cụ thể.
- Leaf Node: Nút lá, không tiếp tục phân chia và chứa các dữ liệu cuối cùng.

Mô hình Random Forest mở rộng ý tưởng này bằng cách sử dụng phương pháp *Bagging*, trong đó mỗi cây được huấn luyện trên một *bộ dữ liệu Bootstrapped* (mẫu ngẫu nhiên có hoàn lại) của dữ liệu gốc.

Cho bộ dữ liệu N mẫu, n đặc trưng là $X = \{(x_i, y_i)\}_{i=1}^{N}$ với $x_i \in \mathbb{R}^n$ và có C lớp sao cho $y_i \in \{1, 2, \dots, C\}$. Bộ dữ liệu Bootstrapped (trung bình sẽ giữ lại 63.2% thông tin gốc[26]) của cây quyết định T_k được định nghĩa:

$$X_k^{(b)} = \{(x_i, y_i)^{(b)}\}_{i=1}^{N} \quad ; \quad (x_i, y_i)^{(b)} \sim \text{wrf } X$$

"wrf" hay *"with replacement from"* ở đây có nghĩa là $X_k^{(b)}$ có thể chứa các mẫu trùng nhau và có thể bỏ qua một số mẫu từ X một cách ngẫu nhiên, ngoài ra thứ tự mẫu cũng sẽ khác hoàn toàn so với X. Nếu ta coi mỗi bộ dữ liệu bất kì tại một Node của cây quyết định T_k là X_{Node}, khi đó mỗi Node sẽ chia bộ dữ liệu thành 2 bộ dữ liệu nhỏ hơn:

$$X_L = \{(x_i, y_i) \mid x_{i,j} \le t\} \quad : \text{ Left split}$$

$$X_R = \{(x_i, y_i) \mid x_{i,j} > t\} \quad : \text{ Right split}$$

Trong đó $j_h \in \{1, \dots, n\}$ là số chỉ của 1 đặc trưng bất kì, $h \in \{1, 2, \dots \lfloor \log_2 n \rfloor\}$ và $j_h \neq j_l \forall h \neq l$. Việc chỉ xét tổng số $\lfloor \log_2 n \rfloor$ đặc trưng ở mỗi bước giúp giảm thời gian tính toán và giảm hiện tượng Overfitting. t là một giá trị ngưỡng phụ thuộc vào đặc trưng j. Mỗi Node sẽ tìm ra phép chia tối ưu nhất bằng cách xét với các giá trị ngưỡng và các đặc trưng khác nhau.

7.1.4 Độ tinh khiết Gini

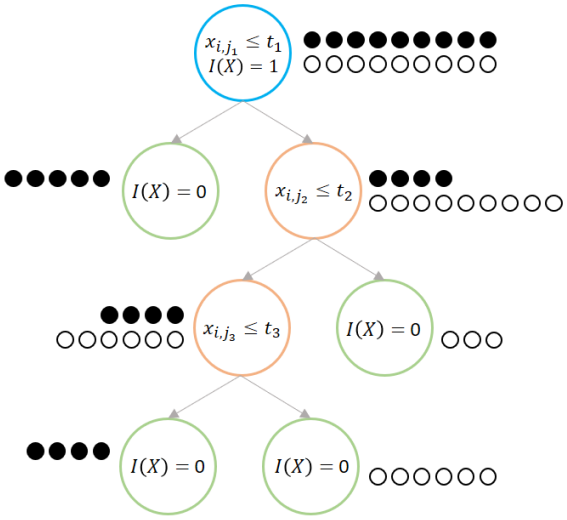
Để có thể tối ưu mỗi bước chia dữ liệu, hay nói cách khác là để các phép chia có thể phân tách các lớp tốt nhất, **độ tinh khiết Gini** được sử dụng:[24]

$$I(X) = 1 - \left(\sum_{c \in \{1,2,\ldots,C\}} \left(\frac{|X_c|}{|X|} \right)^2 \right)$$

Trong đó X_c thể hiện các mẫu trong X thuộc vào lớp c. Ta có hàm mục tiêu:[29]

$$\hat{I} = \arg\min_{j,t} \left(\frac{|X_L|}{|X_{\text{Node}}|} I(X_L) + \frac{|X_R|}{|X_{\text{Node}}|} I(X_R) \right)$$

Với tất cả các nội dung trên, ta có thể hình dung một cây quyết định trong một mô hình RF như sau:



Hình 14: Minh họa một cây quyết định đơn giản

Giá trị Gini đạt tối đa khi dữ liệu phân bố đều giữa các lớp và đạt tối thiểu khi dữ liệu hoàn toàn phân tách. Trong ví dụ trên, cây quyết định với độ sâu 4 đã phân tách dữ liệu thành các *Nút lá thuần khiết (Pure Leaf Node)* khi mà các Node đó chỉ chứa 1 lớp duy nhất. Khi gặp dữ liệu mới, cây sẽ lần lượt so sánh đặc trưng với các ngưỡng tại từng Node cho đến khi đạt đến một Leaf Node, nơi nhãn được gán bởi biểu quyết đa số.

Với các bộ dữ liệu phức tạp, việc phân tách hoàn toàn thường không khả thi hoặc rất tốn kém. Để giải quyết vấn đề đó, ta có thể giới hạn độ sâu cây, đặt ngưỡng tối thiểu cho độ tinh khiết của Leaf Node hoặc các điều kiện dừng khác. Những giới hạn này không chỉ đảm bảo cây được xây dựng hợp lý mà còn giúp giảm hiện tượng Overfitting.

7.1.5 Cách mô hình đưa ra dự đoán

Mô hình RF đưa ra dự đoán cuối cùng bằng cách tổng hợp kết quả từ tất cả các cây quyết định. Đối với bài toán phân loại, lớp của đầu vào x_q được xác định bằng phương pháp biểu quyết đa số:

$$y_q = \text{mode}(\{T_1(x_q), T_2(x_q), \ldots, T_K(x_q)\})$$

Như ta thấy, cả quá trình tạo ra và sử dụng bộ dữ liệu Bootstrapped hay còn gọi là quá trình Bootstrapping về bản chất có sự ngẫu nhiên trong việc chọn mẫu và chọn đặc trưng, do vậy mỗi cây quyết định thường sẽ có những cấu trúc khác nhau. Do sử dụng Bootstrapping và tổng hợp (Aggregating) kết quả của tất cả các cây quyết định, mô hình RF như trên được gọi là *Bagging Random Forest* (**B**ootstrap **AGG**regat**ING**).

7.2 Áp dụng mô hình và kết quả

Sau đây, chúng ta áp dụng mô hình Random Forest với các số cây quyết định và độ sâu khác nhau cho cả bộ dữ liệu TF-IDF và bộ dữ liệu LSA:

7.2.1 Chạy trên bộ dữ liệu TF-IDF

Trees	Max Depth	Train Accuracy	Test Accuracy	F1 Score	Precision	Recall	Fit Time (s)	Test Time (s)
50	10	0.846	0.836	0.855	0.831	0.880	0.232	0.101
	50	0.882	0.867	0.881	0.864	0.899	0.440	0.145
	100	0.888	0.873	0.887	0.864	0.911	0.516	0.166
	nan	0.889	0.874	0.888	0.866	0.912	0.489	0.159
100	10	0.846	0.844	0.862	0.839	0.886	0.453	0.176
	50	0.884	0.870	0.884	0.868	0.901	0.882	0.286
	100	0.891	0.875	0.889	0.866	0.913	0.970	0.289
	nan	0.892	0.876	0.890	0.867	0.914	0.963	0.287

Hình 15: RF với các tham số khác nhau trên bộ dữ liệu TF-IDF

7.2.2 Chạy trên bộ dữ liệu LSA

Trees	Max Depth	Train Accuracy	Test Accuracy	F1 Score	Precision	Recall	Fit Time (s)	Test Time (s)
50	10	0.902	0.863	0.879	0.852	0.908	3.499	0.072
	50	0.941	0.870	0.886	0.854	0.920	4.473	0.114
	100	0.941	0.870	0.886	0.854	0.920	4.702	0.103
	nan	0.941	0.870	0.886	0.854	0.920	4.856	0.114
100	10	0.905	0.866	0.882	0.855	0.910	7.212	0.146
	50	0.942	0.873	0.889	0.856	0.924	8.958	0.207
	100	0.942	0.873	0.889	0.856	0.924	9.070	0.209
	nan	0.942	0.873	0.889	0.856	0.924	8.952	0.204

Hình 16: RF với các tham số khác nhau trên bộ dữ liệu LSA

7.3 Đánh giá mô hình

7.3.1 Nhận xét kết quả

Ta thấy các mô hình Random Forest có kết quả khá tốt, tuy không có vượt trội so với K-NN. Hầu hết các mô hình đều cho hiệu suất tốt hơn khi tăng số cây quyết định và tăng độ sâu cho phép. Phần *"nan"* trong biểu đồ có

nghĩa là ta không giới hạn độ sâu trong các cây quyết định, bắt buộc tất cả các nút lá phải là nút là thuần khiết. Tuy nhiên điều này cũng có vẻ dẫn đến hiện tượng Overfitting khi mà Train Accuracy tăng khá nhanh.

Đặc biệt đối với bộ dữ liệu LSA, hiện tượng Overfitting xảy ra rõ hơn rất nhiều so với bộ dữ liệu TF-IDF, cho thấy rằng Random Forest khá dễ Overfit đối với các bộ dữ liệu dày hơn. Ngoài ra, do bản chất thưa dày của các bộ dữ liệu mà Fit Time của bộ dữ liệu LSA cũng cao hơn so với bộ dữ liệu TF-IDF rất nhiều khi mà việc chọn đặc trưng để chia trở nên đắt đỏ hơn với một bộ dữ liệu dày.

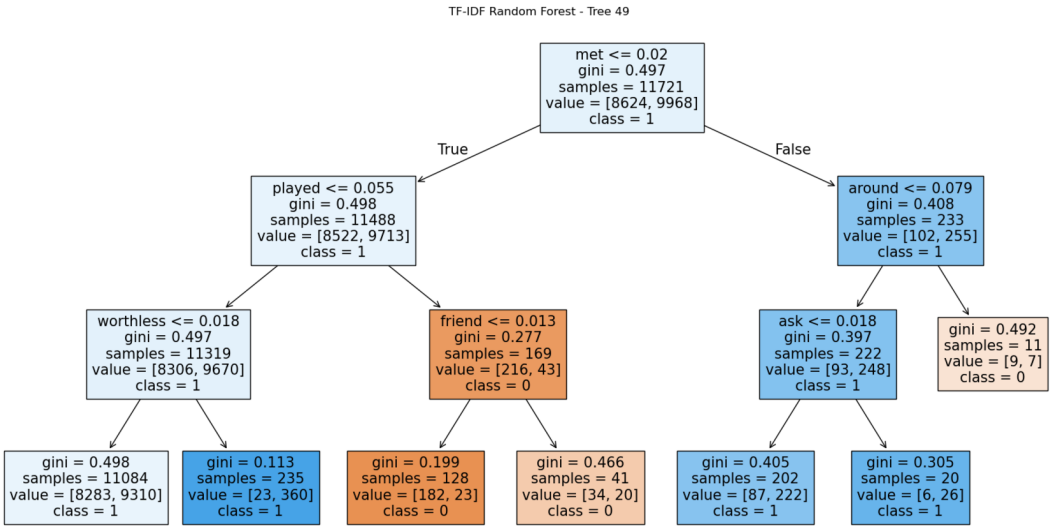
7.3.2 Mô hình tốt nhất

Model	Train Acc.	Test Acc.	F1 Score	Precision	Recall	Fit Time (s)
RF (TF-IDF)	0.892	0.876	0.890	0.867	0.914	0.963
RF (LSA)	0.942	0.873	0.889	0.856	0.924	8.952

Bảng 16: Các mô hình Random Forest tốt nhất theo điểm F1

7.3.3 Giải thích mô hình

Đối với các cây quyết định, một cách vô cùng đơn giản để giải thích chúng là tạo *biểu đồ cây*. Sau đây là một cây quyết định với độ sâu bằng 3 được sử dụng như ví dụ để dễ dàng trực quan:



Hình 17: Trực quan một cây quyết định trên bộ dữ liệu TF-IDF

Ta thấy rõ cách cây quyết định này chia bộ dữ liệu theo các đặc trưng cụ thể ứng với các Tokens, cây quyết định này sẽ xét xem trọng số TF-IDF đối với các Tokens cụ thể trong mẫu văn bản đầu vào và liên tục quyết định xem điểm dữ liệu đó sẽ rơi vào Node nào tiếp theo. Ta có thể dễ dàng hình dung điều tương tự với các mô hình sử dụng bộ dữ liệu LSA, các đặc trưng được xét sẽ được thay bằng các chủ đề tiềm ẩn và mỗi Internal Node sẽ xét độ quan trọng đối với chủ đề đó của mẫu văn bản đầu vào.

Tuy nhiên, do Random Forest là tập hợp nhiều cây quyết định, ta có thể hiểu rằng ta đã đánh đổi khả năng giải thích dễ dàng của một cây quyết định cho khả năng tổng quát hóa tốt hơn và tránh Overfitting hiệu quả hơn.

Chương 8: Phân loại bằng mô hình Logistic Regression

8.1 Khái quát phương pháp

Logistic Regression là một mô hình giám sát thuộc nhóm mô hình tuyến tính giải các bài toán phân loại. Có hai loại Logisitc Regression phổ biến nhất đó là *Multinomial Logistic Regression (Softmax Regression)* giải các bài toán phân loại đa lớp và *Binary Logistic Regression* giải các bài toán phân loại nhị phân.

Logistic Regression là một mô hình vô cùng phổ biến do tính hiệu quả, linh hoạt và khả năng giải thích dễ dàng cho các bài toán phân loại.

Sau đây, chúng ta sẽ xây dựng mô hình Multinomial Logisitc Regression sau đó xét mô hình Binary Logisitc Regression là một trường hợp đặc biệt của Logistic Regression cho bài toán phân loại nhi phân.

8.2 Tổng quan mô hình Multinomial Logistic Regression

8.2.1 Tổng quan mô hình và hàm Softmax

Cho bộ dữ liệu $X = \{(x_i, y_i)\}_{i=1}^{N}$ trong đó $x_i \in \mathbb{R}^n$ và $y_i \in \{1, 2, \ldots, C\}$, vậy đây là một bài toán phân loại đa lớp. Mô hình Multinomial Logistic Regression cho rằng xác suất của đầu vào x thuộc vào lớp $y = c$ là:[10]

$$P(y = c \mid x, \Theta, \mathcal{B}) = \frac{\exp(z_c)}{\sum_{\kappa=1}^{C} \exp(z_\kappa)}; \quad z_\kappa = x^\top \theta_\kappa + \beta_\kappa$$

Trong đó:

- $\Theta = \{\theta_1, \theta_2, \ldots, \theta_C\}$ được gọi là các trọng số và $\mathcal{B} = \{\beta_1, \beta_2, \ldots, \beta_C\}$ được gọi là các hệ số lệch. Chúng tạo nên các tham số của mô hình.
- Tập $Z = \{z_1, z_2, \ldots, z_C\}$ đại diện cho các *Logits* của từng lớp từ 1 đến C, các phẩn tử trong Z là các tổ hợp tuyến tính của đầu vào x và các tham số của mô hình.

Phần biểu thức:

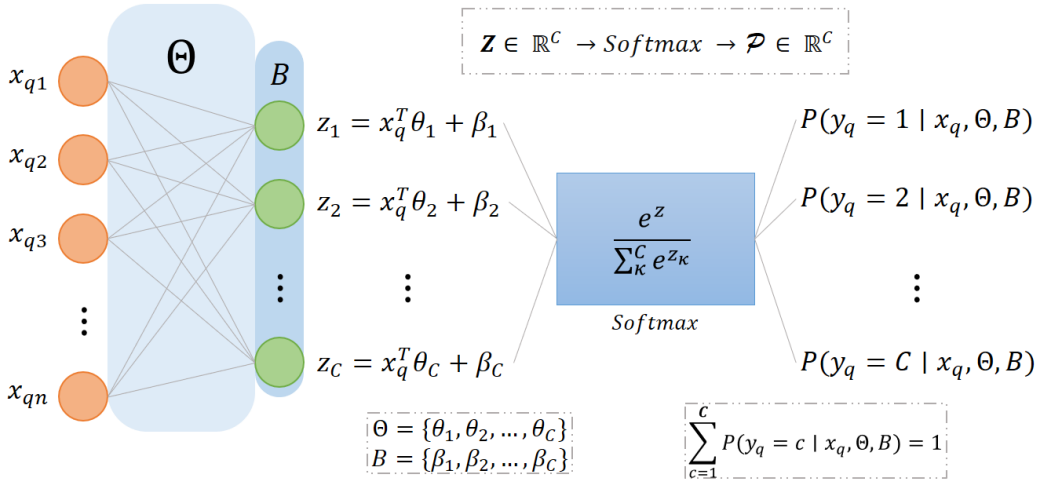
$$\frac{\exp(z_c)}{\sum_{\kappa=1}^{C} \exp(z_\kappa)}$$

Là phần tử thứ c trong $\mathcal{P} = \{P(y = 1 \mid x, \Theta, \mathcal{B}), \ldots, P(y = C \mid x, \Theta, \mathcal{B})\}$ được tạo ra bởi hàm Softmax. Hàm Softmax nhận vào tập Logits $Z \in \mathbb{R}^C$ và tạo ra phân bổ xác suất $\mathcal{P} \in \mathbb{R}^C$ sao cho:

$$\sum_{c=1}^{C} P(y = c \mid x, \Theta, \mathcal{B}) = 1$$

Vậy ta hiểu đơn giản rằng hàm Softmax nhận vào một Vector số thực và tạo ra một phân bổ xác suất có cùng số phần tử với Vector đầu vào.

Sau đây là phần minh họa trực quan mô hình Multinomial Logistic Regression cho đầu vào cần xét x_q:



Hình 18: Trực quan mô hình Multinomial Logistic Regression

Để đưa ra dự đoán, ta đơn giản chọn $y_q = \kappa$ mà ở đó $P(y_q = \kappa \mid x_q, \Theta, \mathcal{B})$ là lớn nhất.

8.2.2 Hàm tổn thất

Mục tiêu của mô hình là tối ưu các tham số Θ và $\mathcal{B}$ cho *hàm hợp lí cực đại (MLE - Maximum Likelihood Estimation)*:[10,19]

$$L(\Theta, \mathcal{B}) = \prod_{i=1}^{N} \prod_{c=1}^{C} \left(\frac{\exp(x_i^\top \theta_c + \beta_c)}{\sum_{\kappa=1}^{C} \exp(x_i^\top \theta_\kappa + \beta_\kappa)} \right)^{1[y_i = c]}$$

Trong đó $1[y_i = c]$ là một *hàm chỉ thị* và bằng 1 nếu $y_i = c$ và bằng 0 trong các trường hợp khác.

Để đơn giản hóa tính toán, ta xét hàm Logarit hợp lí:

$$\log L(\Theta, \mathcal{B}) = \sum_{i=1}^{N} \sum_{c=1}^{C} 1[y_i = c] \log \left(\frac{\exp(x_i^\top \theta_c + \beta_c)}{\sum_{\kappa=1}^{C} \exp(x_i^\top \theta_\kappa + \beta_\kappa)} \right)$$

$$= \sum_{i=1}^{N} x_i^\top \theta_c + \beta_c - \log \left(\sum_{\kappa=1}^{C} \exp(x_i^\top \theta_\kappa + \beta_\kappa) \right)$$

Việc cực đại hóa hàm MLE tương đương với việc cực tiểu hóa hàm Logarit hợp lí với dấu âm, từ đó ta định nghĩa hàm tổn thất:

$$J(\Theta, \mathcal{B}) = -\frac{1}{N} \log L(\Theta, \mathcal{B})$$

8.2.3 Thuật toán Gradient Descent

Do việc trực tiếp giải bài toán tối ưu trên là gần như không thể, ta thường sử dụng các phương pháp giải lặp. Một trong các thuật toán phổ biến và hiệu quả nhất để giải bài toán tối ưu của Logistic Regression chính là *Gradient Descent*.

Gradient Descent tận dụng đạo hàm riêng của các tham số của mô hình đối với hàm tổn thất để tìm ra một cực tiểu địa phương của hàm tổn thất. Ta có thể hiểu rằng trong không gian của hàm tổn thất, Gradient Descent

đang tìm ra hướng của các tham số đại diện bởi một điểm mà ở đó hàm tổn thất sẽ giảm tới một điểm cực tiểu địa phương nhanh nhất.

Các bước thuật toán:[10,19]

Bước 1: Khởi tạo tham số $\Theta, \mathcal{B}$ ngẫu nhiên.

Bước 2: Ta tính các đạo hàm riêng của $J(\Theta, \mathcal{B})$ đối với các bộ tham số của từng Logit gồm θ_c và β_c:

$$\frac{\partial J}{\partial \theta_c} = -\frac{1}{N}\sum_{i=1}^{N}\left(1[y_i = c] - \frac{\exp(x_i^\top \theta_c + \beta_c)}{\sum_{\kappa=1}^{C}\exp(x_i^\top \theta_\kappa + \beta_\kappa)}\right)x_i$$

$$\frac{\partial J}{\partial \beta_c} = -\frac{1}{N}\sum_{i=1}^{N}\left(1[y_i = c] - \frac{\exp(x_i^\top \theta_c + \beta_c)}{\sum_{\kappa=1}^{C}\exp(x_i^\top \theta_\kappa + \beta_\kappa)}\right)$$

Bước 3: Cập nhật các tham số cho từng Logit:

$$\theta_c^{\text{new}} = \theta_c - \alpha\frac{\partial J}{\partial \theta_c}$$

$$\beta_c^{\text{new}} = \beta_c - \alpha\frac{\partial J}{\partial \beta_c}$$

Bước 4: Kiểm tra hội tụ:

$$|\Delta J| = |J(\Theta^{\textbf{new}}, \mathcal{B}^{\textbf{new}}) - J(\Theta, \mathcal{B})| < \epsilon$$

Trong đó α được gọi là *hệ số học*, là độ lớn bước nhảy của thuật toán tới một cực tiểu địa phương.

8.2.4 L2 Regularization

Đối với Logisitc Regression, ta thường có thể thêm phần *Regularization* để giảm hiện tượng Overfitting. Ta sẽ tập trung vào một dạng chính trong báo cáo này đó là *L2 Regularization*.

Ta có thể thêm phần L2 Regularization vào mô hình của chúng ta bằng cách điều chỉnh hàm tổn thất:[10,19]

$$J_{\text{reg}}(\Theta, \mathcal{B}) = -\frac{1}{N}\log L(\Theta, \mathcal{B}) + \frac{\lambda}{2N}\sum_{c=1}^{C}\|\theta_c\|_2^2$$

Trong đó λ là một hằng số kiểm soát hiệu ứng của Regularization. Các trọng số có phân bổ gần 0 hơn khi λ lớn ($\lambda > 1$), giúp giảm hiện tượng Overfitting:

$$\frac{\partial J_{\text{reg}}}{\partial \theta_c} = -\frac{1}{N}\sum_{i=1}^{N}\left(1[y_i = c] - \frac{\exp(x_i^\top \theta_c + \beta_c)}{\sum_{\kappa=1}^{C}\exp(x_i^\top \theta_\kappa + \beta_\kappa)}\right)x_i + \frac{\lambda}{N}\theta_c$$

$$\theta_c^{\text{new}} = \theta_c - \alpha\frac{\partial J_{\text{reg}}}{\partial \theta_c}$$

8.3 Tổng quan mô hình Binary Logistic Regression

Khi chỉ có $C = 2$ lớp, bài toán phân loại trở thành bài toán phân loại nhị phân, để đơn giản ta cho $y_i \in \{0, 1\}$. Khi đó hàm Softmax trở thành một trường hợp đặc biệt gọi là hàm Sigmoid:[37]

$$\frac{1}{\exp(x_i^\top \theta_1 + \beta_1) + \exp(x_i^\top \theta_2 + \beta_2)} \begin{bmatrix} \exp(x_i^\top \theta_1 + \beta_1) \\ \exp(x_i^\top \theta_2 + \beta_2) \end{bmatrix}$$

$$= \frac{1}{\exp((\theta_1 - \theta_2)x_i^\top + \beta_1 - \beta_2) + \exp((\vec{0})x_i^\top)} \begin{bmatrix} \exp((\theta_1 - \theta_2)x_i^\top + \beta_1 - \beta_2) \\ \exp((\vec{0})x_i^\top) \end{bmatrix}$$

$$= \begin{bmatrix} \frac{\exp((\theta_1-\theta_2)x_i^\top+\beta_1-\beta_2)}{1+\exp((\theta_1-\theta_2)x_i^\top+\beta_1-\beta_2)} \\ \frac{1}{1+\exp((\theta_1-\theta_2)x_i^\top+\beta_1-\beta_2)} \end{bmatrix} = \begin{bmatrix} 1 - \frac{1}{1+\exp((\theta_1-\theta_2)x_i^\top+\beta_1-\beta_2)} \\ \frac{1}{1+\exp((\theta_1-\theta_2)x_i^\top+\beta_1-\beta_2)} \end{bmatrix}$$

Đặt $\theta' = \theta_2 - \theta_1$ và $\beta' = \beta_2 - \beta_1$, ta có:

$$\frac{1}{1 + \exp(-(x_i^\top \theta' + \beta'))} \quad ; \quad 1 - \frac{1}{1 + \exp(-(x_i^\top \theta' + \beta'))}$$

Trong đó, phương trình bên trái chính là hàm Sigmoid và biểu diễn xác suất của đầu vào x_i thuộc vào lớp 1 và phần bên phải biểu diễn xác suất của đầu vào x_i thuộc vào lớp 0.

Như vậy, khi $C = 2$, ta chỉ cần tối ưu hóa một bộ trọng số θ' và một hệ số lệch β'. Cho bộ dữ liệu $X = \{(x_i, y_i)\}_{i=1}^N$ trong đó $x_i \in \mathbb{R}^n$ và $y_i \in \{0, 1\}$. Mô hình Binary Logistic Regression hay đơn giản là Logistic Regression được mô hình hóa:[8,37]

$$P(y \mid x, \theta, \beta) = (\sigma(z))^y (1 - \sigma(z))^{1-y}; \quad z = x^\top \theta' + \beta'$$

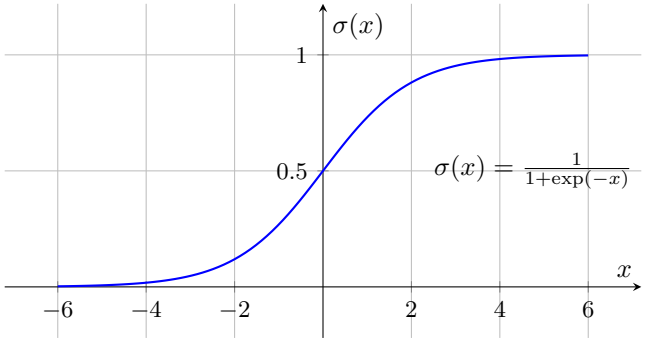
Trong đó:

- Các trọng số $\theta' = \{\theta_1, \theta_2, \ldots, \theta_n\}$ và hệ số lệch $\beta' \in \mathbb{R}$ là các tham số của mô hình.

Phần biểu thức:

$$\sigma(z) = \frac{1}{1 + \exp(-z)}$$

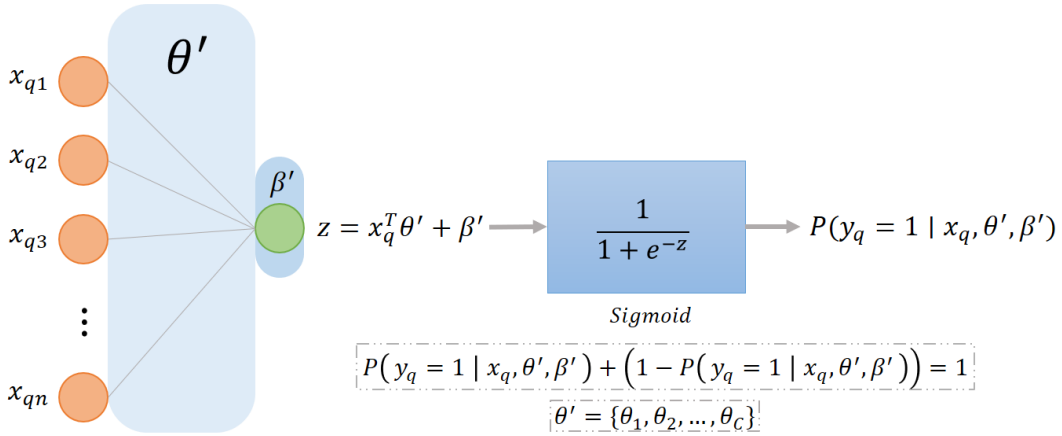
Là hàm Sigmoid và $z = x^\top \theta' + \beta'$ là tổ hợp tuyến tính giữa đầu vào x và các tham số của mô hình. Hàm Sigmoid có một vài đặc điểm nổi bật: tiến tới 0 khi $z \to -\infty$ và tiến tới 1 khi $z \to +\infty$ và $\sigma(z) + (1 - \sigma(z)) = 1$.



Hình 19: Hàm Sigmoid

Vậy, hàm Sigmoid nhận vào đầu vào là một số thực và trả về một số thực $\sigma(x)$ nằm trong khoảng $[0, 1]$ đại diện cho xác suất, để có một phân bổ gồm hai xác suất có tổng là 1 ta đơn giản lấy $\sigma(x)$ và $1 - \sigma(x)$.

Sau đây là phần minh họa trực quan mô hình Binary Logistic Regression cho đầu vào cần xét x_q:



Hình 20: Trực quan mô hình Binary Logistic Regression

Để đưa ra dự đoán, ta đơn giản xét một ngưỡng t, thường bằng 0.5, x_q sẽ thuộc lớp 1 nếu $P(y_q = 1 \mid x_q, \theta', \beta') > t$ và thuộc lớp 0 nếu ngược lại.

Lúc này, các tính toán của chúng ta sẽ khác đi một chút:[8, 17, 19]

MLE:

$$L(\theta', \beta') = \prod_{i=1}^{N} P(y_i \mid x_i, \theta', \beta')$$

Hàm Logarit hợp lí:

$$\log L(\theta', \beta') = \sum_{i=1}^{N} [y_i \log \sigma(z_i) + (1 - y_i) \log(1 - \sigma(z_i))]$$

Hàm tổn thất:

$$J(\theta', \beta') = -\frac{1}{N} \log L(\theta', \beta')$$

Gradient Descent:

$$\frac{\partial J}{\partial \theta'} = -\frac{1}{N} \sum_{i=1}^{N} (\sigma(z_i) - y_i) x_i \quad ; \quad \frac{\partial J}{\partial \beta'} = -\frac{1}{N} \sum_{i=1}^{N} (\sigma(z_i) - y_i)$$

$$\theta'^{\text{ new}} = \theta' - \alpha \frac{\partial J}{\partial \theta'} \quad ; \quad \beta'^{\text{ new}} = \beta' - \alpha \frac{\partial J}{\partial \beta'}$$

L2 Regularization:

$$J_{\text{reg}}(\theta', \beta') = -\frac{1}{N} \log L(\theta', \beta') + \frac{\lambda}{2N} \|\theta'\|_2^2$$

$$\frac{\partial J_{\text{reg}}}{\partial \theta'} = -\frac{1}{N} \sum_{i=1}^{N} (\sigma(z_i) - y_i) x_i + \frac{\lambda}{N} \theta' \quad ; \quad \theta'^{\text{ new}} = \theta' - \alpha \frac{\partial J_{\text{reg}}}{\partial \theta'}$$

8.4 Áp dụng mô hình và kết quả

Sau đây, chúng ta áp dụng mô hình Logistic Regression, không có và có sử dụng L2 Regularization với các tham số Regularization khác nhau cho cả bộ dữ liệu TF-IDF và bộ dữ liệu LSA:

8.4.1 Chạy trên bộ dữ liệu TF-IDF

C	Train Accuracy	Test Accuracy	F1 Score	Precision	Recall
None	0.939	0.923	0.930	0.930	0.929
0.010	0.879	0.878	0.893	0.865	0.922
0.100	0.910	0.903	0.911	0.915	0.908
5	0.953	0.925	0.932	0.934	0.930
10	0.961	0.921	0.927	0.930	0.924
100	0.977	0.900	0.909	0.911	0.906

Hình 21: Logistic Regression với các tham số khác nhau trên bộ dữ liệu TF-IDF

8.4.2 Chạy trên bộ dữ liệu LSA

C	Train Accuracy	Test Accuracy	F1 Score	Precision	Recall
None	0.910	0.908	0.916	0.920	0.912
0.010	0.875	0.873	0.888	0.860	0.918
0.100	0.898	0.898	0.907	0.913	0.901
5	0.912	0.910	0.918	0.922	0.914
10	0.912	0.910	0.918	0.922	0.914
100	0.912	0.911	0.918	0.923	0.914

Hình 22: Logistic Regression với các tham số khác nhau trên bộ dữ liệu LSA

8.5 Đánh giá mô hình

8.5.1 Nhận xét kết quả

Logistic Regression đã thể hiện hiệu suất vượt trội trong các thử nghiệm, đặc biệt trên bộ dữ liệu TF-IDF với điểm F1 đạt 0.932 khi $C = 5$. Ngay cả khi không sử dụng Regularization ($C = None$), mô hình vẫn duy trì hiệu suất cao với điểm F1 là 0.930. Điều này cho thấy dữ liệu đầu vào đã được xử lý hiệu quả, đảm bảo khả năng thu nhận các mối quan hệ tuyến tính trong dữ liệu của mô hình Logistic Regression.

Khi tăng tham số C lên đến giá trị lớn như $C = 100$, độ chính xác trên tập huấn luyện tăng nhưng hiệu suất trên tập kiểm tra giảm, là một biểu hiện của hiện tượng Overfitting. Ngược lại, giá trị C nhỏ (ví dụ, $C = 0.1$) dẫn đến sự cân bằng tốt hơn giữa tập huấn luyện và tập kiểm tra. Điều này nhấn mạnh vai trò quan trọng của việc điều chỉnh Regularization để tối ưu hóa hiệu suất.

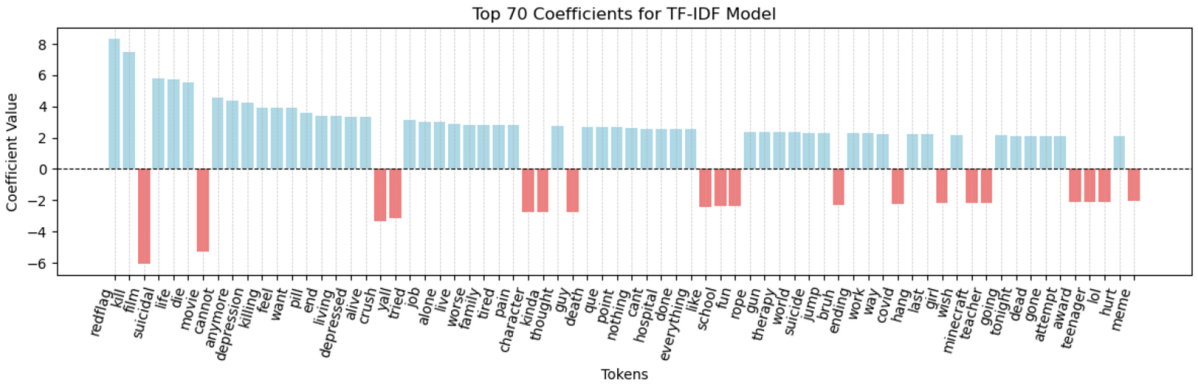
Đối với bộ dữ liệu LSA, mặc dù hiệu suất thấp hơn TF-IDF, sự chênh lệch không đáng kể cho thấy LSA là phương pháp giảm chiều hiệu quả, duy trì phần lớn thông tin quan trọng. Như vậy, ta thấy Logistic Regression là một lựa chọn tối ưu với cả hai bộ dữ liệu, nhờ vào sự ổn định và khả năng khai thác tốt các đặc trưng giúp cho nhiệm vụ giải thích mô hình.

8.5.2 Mô hình tốt nhất

Model	Train Acc.	Test Acc.	F1 Score	Precision	Recall
L2 Log.Reg. C=5 (TF-IDF)	0.953	0.925	0.932	0.934	0.930
L2 Log.Reg. C=5 (LSA)	0.912	0.910	0.918	0.922	0.914

Bảng 17: Các mô hình Logistic Regression tốt nhất theo điểm F1

8.5.3 Giải thích mô hình



Hình 23: Các tham số của mô hình Log. Reg. trên bộ dữ liệu TF-IDF

Khả năng giải thích của Logistic Regression đến từ các trọng số, đại diện cho mức độ quan trọng của từng đặc trưng trong việc dự đoán nhãn.[27] Với bộ dữ liệu TF-IDF, các trọng số dương thường tương ứng với các Tokens nghiêng về lớp 1, trong khi trọng số âm thường đại diện cho lớp 0. Điều này giúp chúng ta hiểu rõ cách các từ trong văn bản ảnh hưởng đến phân loại.

Trên bộ dữ liệu LSA, các trọng số không còn gắn liền với các Tokens cụ thể mà liên quan đến các chủ đề tiềm ẩn. Mặc dù việc giải thích trở nên trừu tượng hơn, các trọng số này vẫn cung cấp thông tin quan trọng về vai trò của từng chủ đề trong bài toán phân loại. Đặc biệt khi kết hợp với các phương pháp phân tích chủ đề tiềm ẩn trong những phần trước, ta có được tiềm năng khai thác phân tích mạnh mẽ từ các chủ đề tiềm ẩn.

Nhìn chung, khả năng giải thích của Logistic Regression không chỉ hỗ trợ việc đánh giá hiệu suất mà còn giúp hiểu rõ hơn về cấu trúc dữ liệu và các yếu tố quyết định trong phân loại, làm tăng giá trị ứng dụng của mô hình.

Tổng hợp kết quả phân loại

Model	Train Acc.	Test Acc.	F1 Score	Precision	Recall	Fit Time (s)	Test Time (s)
Euclidean K-NN K=50 (TF-IDF)	0.889	0.888	0.887	0.892	0.888	-	2.867
Cosine K-NN K=10 (LSA)	0.891	0.866	0.865	0.866	0.866	-	1.191
RF (TF-IDF)	0.892	0.876	0.890	0.867	0.914	0.963	-
RF (LSA)	0.942	0.873	0.889	0.856	0.924	8.952	-
L2 Log.Reg. C=5 (TF-IDF)	0.953	0.925	0.932	0.934	0.930	-	-
L2 Log.Reg. C=5 (LSA)	0.912	0.910	0.918	0.922	0.914	-	-

Bảng 18: Tổng hợp kết quả quá trình phân loại

Ở bảng trên, các mô hình tối ưu nhất được lựa chọn dựa trên F1 Score. Ta có một số nhận xét:

- **Logistic Regression (L2 Regularization)**: Hiệu suất cao nhất với F1 Score 0.932 (TF-IDF) và 0.918 (LSA), kèm thời gian huấn luyện và kiểm tra thấp nhất, cho thấy tính vượt trội cả về độ chính xác lẫn hiệu quả tính toán.
- **Random Forest (RF)**: Điểm F1 tốt (TF-IDF: 0.890, LSA: 0.889), nhưng thời gian huấn luyện trên LSA (8.952 giây) cao hơn nhiều, cho thấy RF nhạy cảm với kích thước dữ liệu.
- **K-NN (Euclidean và Cosine)**: Hiệu suất thấp hơn Logistic Regression (TF-IDF: 0.887, LSA: 0.865) và thời gian kiểm thử cao (TF-IDF: 2.867 giây, LSA: 1.191 giây). LSA giúp giảm thời gian tính toán nhưng làm giảm nhẹ độ chính xác.
- **So sánh giữa TF-IDF và LSA**:
  - Hiệu suất các mô hình khi sử dụng LSA giảm không đáng kể, đặc biệt là Logistic Regression (giảm chỉ 1.4% ở F1 Score). Điều này cho thấy LSA đã giữ lại phần lớn thông tin quan trọng từ dữ liệu gốc.
  - Tuy nhiên, đối với RF và K-NN, việc giảm chiều dữ liệu giúp cải thiện đáng kể thời gian tính toán nhưng cũng làm giảm nhẹ độ chính xác.
- **Kết luận tổng thể**:
  - Logistic Regression là mô hình nổi bật nhất, vượt trội về cả hiệu suất (F1 Score, Precision, Recall) và chi phí tính toán, làm rõ tầm quan trọng của việc xử lý và giảm chiều dữ liệu trong bài toán phân loại.
  - Random Forest và K-NN thể hiện hiệu quả trên các đặc thù dữ liệu khác nhau, nhưng chi phí tính toán cao khiến chúng kém phù hợp hơn trong bối cảnh yêu cầu tốc độ xử lý nhanh.
  - LSA là một phương pháp giảm chiều dữ liệu hiệu quả, đặc biệt khi kết hợp với Logistic Regression, giúp cân bằng giữa hiệu suất và chi phí tính toán.

Chương 9: Kết luận

9.1 Tóm tắt kết quả và Nhận xét chung

Báo cáo đã sử dụng TF-IDF để chuyển đổi dữ liệu văn bản thành dạng bảng, giúp các mô hình và phương pháp học máy hoạt động hiệu quả, dù hạn chế về việc không nắm bắt được ngữ nghĩa và ngữ cảnh sâu. LSA được áp dụng để khám phá cấu trúc chủ đề tiềm ẩn trong dữ liệu, vừa giảm chiều vừa giữ được tính trung thực với bộ Corpus gốc, giúp tăng hiệu quả tính toán. Phân cụm bằng K-Means và GMM cung cấp cái nhìn tổng quan về cấu trúc tiềm ẩn, hỗ trợ phỏng đoán các chủ đề trong văn bản.

Trong phần phân loại, K-NN, Random Forest, và Logistic Regression được thử nghiệm, với Logistic Regression nổi bật nhất về hiệu suất. So sánh giữa TF-IDF và LSA cũng cho thấy hiệu quả của việc giảm chiều dữ liệu trong việc tối ưu hóa mô hình. Nhìn chung, kết quả đã khẳng định tiềm năng của việc kết hợp các phương pháp phân tích và mô hình học máy trong chủ đề đánh giá sức khỏe tâm lý.

9.2 Hạn chế của bài báo cáo

Báo cáo tập trung vào phân tích bộ văn bản về sức khỏe tâm lý nhưng chưa thể khai thác sâu các khía cạnh ngữ nghĩa và ngữ cảnh, điều này có thể làm giảm độ chính xác trong việc nắm bắt ý nghĩa thực sự của văn bản. Cách trích xuất dữ liệu hiện tại vẫn mang tính cơ bản, bỏ qua các tín hiệu ngôn ngữ tinh tế có liên quan đến trạng thái tâm lý. Ngoài ra, một số nhận xét còn mang tính chủ quan và không chuyên sâu. Báo cáo cũng chỉ sử dụng các công nghệ nền tảng, chưa khai thác hết tiềm năng của các phương pháp hiện đại trong xử lý ngôn ngữ và học máy, vốn có thể nắm bắt tốt hơn các biểu hiện tâm lý phức tạp trong văn bản.

9.3 Định hướng nghiên cứu tương lai

Để nâng cao chất lượng nghiên cứu về sức khỏe tâm lý, cần xem xét tích hợp các phương pháp trích xuất dữ liệu tiên tiến hơn, giúp bảo toàn ngữ nghĩa và ngữ cảnh, nhằm phản ánh chính xác hơn các trạng thái tâm lý từ văn bản. Ngoài ra, việc sử dụng các mô hình hiện đại như GPT, BERT, hoặc các mô hình ngôn ngữ lớn khác có thể hỗ trợ hiệu quả trong việc phát hiện các dấu hiệu tinh tế liên quan đến sức khỏe tâm lý. Đây là bước tiến quan trọng để tạo ra các công cụ hỗ trợ tự động, giúp nhận diện và hỗ trợ người cần giúp đỡ một cách chính xác và kịp thời. Việc kết hợp các phương pháp này không chỉ cải thiện hiệu suất mà còn mở rộng phạm vi ứng dụng trong các lĩnh vực liên quan đến phân tích tâm lý và chăm sóc sức khỏe.

Danh sách hình minh họa và bảng

Danh sách hình vẽ

1 Tần suất các mẫu có tổng Tokens trong các khoảng khác nhau . . . 10
2 Các biểu đồ Word Cloud được tạo ra trong quá trình phân tích . . . 11
3 Kết quả phân tích n-grams . . . 12
4 Trực quan kết quả của SVD . . . 20
5 Các mẫu văn bản trong không gian ba chiều được tìm bởi LSA từ nhiều góc nhìn . . . 24
6 Các mẫu văn bản trong không gian hai chiều được tìm bởi LSA . . . 25
7 Biplots chứa các mẫu văn bản và các Vector Tokens trong không gian hai chiều được tìm bởi LSA . . . 26
8 Kết quả phân cụm K-Means với hai cụm . . . 28
9 Kết quả phân cụm K-Means với ba cụm . . . 29
10 Kết quả phân cụm GMM với hai cụm . . . 31
11 Kết quả phân cụm GMM với ba cụm . . . 32
12 K-NN sử dụng Euclidean và Cosine với các giá trị k khác nhau trên bộ dữ liệu TF-IDF . . . 35
13 K-NN sử dụng Euclidean và Cosine với các giá trị k khác nhau trên bộ dữ liệu LSA . . . 35
14 Minh họa một cây quyết định đơn giản . . . 38
15 RF với các tham số khác nhau trên bộ dữ liệu TF-IDF . . . 39
16 RF với các tham số khác nhau trên bộ dữ liệu LSA . . . 39
17 Trực quan một cây quyết định trên bộ dữ liệu TF-IDF . . . 40
18 Trực quan mô hình Multinomial Logistic Regression . . . 42
19 Hàm Sigmoid . . . 44
20 Trực quan mô hình Binary Logistic Regression . . . 45
21 Logistic Regression với các tham số khác nhau trên bộ dữ liệu TF-IDF . . . 46
22 Logistic Regression với các tham số khác nhau trên bộ dữ liệu LSA . . . 46
23 Các tham số của mô hình Log. Reg. trên bộ dữ liệu TF-IDF . . . 47

Danh sách bảng

1 Một vài mẫu văn bản và lớp của chúng trong bộ dữ liệu . . . 8
2 Thông số bộ Corpus gốc và bộ Corpus được làm sạch . . . 10
3 Giải thích về trọng số TF-IDF với các kết hợp giá trị TF và IDF khác nhau . . . 15
4 Các Tokens có tổng phần tử nhỏ nhất trong các tập từ vựng của TF-IDF với các giá trị s khác nhau . . . 16
5 Đặc điểm của một vài Vector văn bản tạo bởi TF-IDF . . . 17
6 Các văn bản có độ tương đồng Cosine lớn nhất với các văn bản được chọn (TF-IDF) . . . 18
7 Các Tokens có độ tương đồng Cosine lớn nhất với các Tokens được chọn lọc (TF-IDF) . . . 18
8 Đặc điểm của một số Vector văn bản trong bộ dữ liệu LSA . . . 21
9 Một vài chủ đề và các Tokens đại diện cho chúng (đã được chọn lọc) . . . 22
10 Các văn bản đại diện cho chủ đề 1 và chủ đề 2 . . . 22
11 Các văn bản có độ tương đồng Cosine lớn nhất với các văn bản được chọn (LSA) . . . 23
12 Các Tokens có độ tương đồng Cosine lớn nhất với các Tokens được chọn lọc (LSA) . . . 23
13 Phân bổ lớp trong tập Train . . . 33
14 Phân bổ lớp trong tập Test . . . 33
15 Các mô hình K-NN tốt nhất theo điểm F1 . . . 36
16 Các mô hình Random Forest tốt nhất theo điểm F1 . . . 40
17 Các mô hình Logistic Regression tốt nhất theo điểm F1 . . . 47
18 Tổng hợp kết quả quá trình phân loại . . . 48

Tài liệu tham khảo

[1] Shereen Albitar, Sébastien Fournier, and Bernard Espinasse. An effective tf/idf-based text-to-text semantic similarity measure for text classification. In *Web Information Systems Engineering–WISE 2014: 15th International Conference, Thessaloniki, Greece, October 12-14, 2014, Proceedings, Part I 15*, pages 105–114. Springer, 2014.

[2] G Aliman, Tanya Faye S Nivera, Jensine Charmille A Olazo, Daisy Jane P Ramos, Chris Danielle B Sanchez, Timothy M Amado, Nilo M Arago, Romeo L Jorda Jr, Glenn C Virrey, and Ira C Valenzuela. Sentiment analysis using logistic regression. *Journal of Computational Innovations and Engineering Applications*, 7(1):35–40, 2022.

[3] Reihaneh Neda Ardjmand. Mental health corpus, 2021. 8

[4] Madan Mohan Tito Ayyalasomayajula, Akshay Agarwal, and Shahnawaz Khan. Reddit social media text analysis for depression prediction: using logistic regression with enhanced term frequency-inverse document frequency features. *International Journal of Electrical & Computer Engineering (2088-8708)*, 14(5), 2024.

[5] Prafulla Bafna, Dhanya Pramod, and Anagha Vaidya. Document clustering: Tf-idf approach. In *2016 International Conference on Electrical, Electronics, and Optimization Techniques (ICEEOT)*, pages 61–66. IEEE, 2016.

[6] Gérard Biau. Analysis of a random forests model. *The Journal of Machine Learning Research*, 13(1):1063–1095, 2012.

[7] Mohamed Chiny, Marouane Chihab, Omar Bencharef, and Younes Chihab. Netflix recommendation system based on tf-idf and cosine similarity algorithms. *no. Bml*, pages 15–20, 2022.

[8] Cao Văn Chung. Binary logistic regression. https://drive.google.com/file/d/1HmzUgyuXd5G8JlrB38nrB2qVdukmXmcQ/view. Informatics Dept., MIM, HUS, VNU Hanoi. 44, 45

[9] Cao Văn Chung. Gaussian mixture model and k-means clustering. https://drive.google.com/file/d/1t7hHaQHDyuYeKwMCx7X8nrr3yxe-euSd/view. Lecture notes, Informatics Dept., MIM, HUS, VNU Hanoi. 30

[10] Cao Văn Chung. Multinomial logistic regression. https://drive.google.com/file/d/1G-MYxShyJQZji_Rx0YcdeaRrgs98o9ra/view. Informatics Dept., MIM, HUS, VNU Hanoi. 41, 42, 43

[11] Mike Conway, Mengke Hu, and Wendy W Chapman. Recent advances in using natural language processing to address public health research questions using social media and consumergenerated data. *Yearbook of medical informatics*, 28(01):208–217, 2019.

[12] Benjamin L Cook, Ana M Progovac, Pei Chen, Brian Mullin, Sherry Hou, and Enrique Baca-Garcia. Novel use of natural language processing (nlp) to predict suicidal ideation and psychiatric symptoms in a text-based mental health intervention in madrid. *Computational and mathematical methods in medicine*, 2016(1):8708434, 2016.

[13] Glen Coppersmith, Ryan Leary, Patrick Crutchley, and Alex Fine. Natural language processing of social media as screening for suicide risk. *Biomedical informatics insights*, 10:1178222618792860, 2018.

[14] G.H. Golub, Alan Hoffman, and G.W. Stewart. A generalization of the eckart-young-mirsky matrix approximation theorem. *Linear Algebra and its Applications*, 88-89:317–327, 1987. 20

[15] Fritz Günther, Carolin Dudschig, and Barbara Kaup. Latent semantic analysis cosines as a cognitive similarity measure: Evidence from priming studies. *Quarterly Journal of Experimental Psychology*, 69(4):626–653, 2016.

[16] Per Hedelin and Jan Skoglund. Vector quantization based on gaussian mixture models. *IEEE transactions on speech and audio processing*, 8(4):385–401, 2000.

[17] Joseph M Hilbe. Logistic regression. *International encyclopedia of statistical science*, 1:15–32, 2011. 45

[18] Diana Inkpen and Atefeh Farzindar. Natural language processing for social media. Based on the tutorial: Applications of Social Media Text Analysis, September 18, 2015, 2017. Available from NLP Technologies Inc. and University of Ottawa.

[19] Daniel Jurafsky and James H. Martin. Speech and language processing: Chapter 5 - logistic regression. https://web.stanford.edu/~jurafsky/slp3/5.pdf, 2024. Draft of August 20, 2024, Copyright ©2024, All rights reserved. 42, 43, 45

[20] Ammar Kadhim. An evaluation of preprocessing techniques for text classification. *International Journal of*

Computer Science and Information Security,, 16:22–32, 06 2018. 9

[21] Khushbu Khamar. Short text classification using knn based on distance function. *International Journal of advanced research in computer and communication engineering*, 2(4):1916–1919, 2013.

[22] Divya Khyani, Siddhartha B S, N. Niveditha, and Dr Y M. An interpretation of lemmatization and stemming in natural language processing. *Shanghai Ligong Daxue Xuebao/Journal of University of Shanghai for Science and Technology*, 22:350–357, 01 2021. 9

[23] Gang Kou and Yi Peng. An application of latent semantic analysis for text categorization. *International Journal of Computers Communications & Control*, 10(3):357–369, 2015.

[24] Eduardo Laber and Lucas Murtinho. Minimization of gini impurity: Np-completeness and approximation algorithm via connections with the k-means problem. *Electronic Notes in Theoretical Computer Science*, 346:567–576, 08 2019. 38

[25] Thomas K Landauer, Peter W Foltz, and Darrell Laham. An introduction to latent semantic analysis. *Discourse processes*, 25(2-3):259–284, 1998.

[26] Venus Khim-Sen Liew. An overview on various ways of bootstrap methods. MPRA Paper 7163, MPRA, April 2016. Subjects: C - Mathematical and Quantitative Methods > C1 - Econometric and Statistical Methods and Methodology: General > C15 - Statistical Simulation Methods: General. 37

[27] A.C. Müller and S. Guido. *Introduction to Machine Learning with Python: A Guide for Data Scientists*, chapter 7. O'Reilly Media, Incorporated, 2018. Focusing on Chapter 7: Working with Text Data. 9, 14, 47

[28] Andrew Ng. Cs229 lecture notes: The em algorithm. https://cs229.stanford.edu. Lecture notes for Stanford's Machine Learning course (CS229). 30

[29] Pavlos Protopapas, Kevin Rader, Rahul Dave, and Margo Levine. Lecture #15: Regression trees & random forests. Data Science 1, CS 109A, STAT 121A, AC 209A, E-109A, Harvard University, 2017. 38

[30] Kristina P.S and Miin-Shen Yang. Unsupervised k-means clustering algorithm. *IEEE Access*, PP:1–1, 04 2020. 27

[31] Evangelos Psomakelis, Konstantinos Tserpes, Dimosthenis Anagnostopoulos, and Theodora Varvarigou. Comparing methods for twitter sentiment analysis. *arXiv preprint arXiv:1505.02973*, 2015.

[32] Douglas A Reynolds et al. Gaussian mixture models. *Encyclopedia of biometrics*, 741(659-663), 2009. 30, 31

[33] David Rosenberg. Em algorithm for latent variable models. https://davidrosenberg.github.io/mlcourse/Archive/2017/Lectures/14a.EM-algorithm.pdf, 2017. Lecture notes, New York University, Machine Learning course, May 2, 2017. 30

[34] Ashish Sharma, Adam S Miner, David C Atkins, and Tim Althoff. A computational approach to understanding empathy expressed in text-based mental health support. *arXiv preprint arXiv:2009.08441*, 2020.

[35] PV Snigdha, M Naveen, and S Rahul. Movie recommendation system using tf-idf vectorization and cosine similarity. *INTERNATIONAL JOURNAL OF ENGINEERING DEVELOPMENT AND RESEARCH*, 10(3):1–8, 2022.

[36] Rebecca C. Steorts. K-means clustering. https://www2.stat.duke.edu/~rcs46/lectures_2017/10-unsupervise/10-kmeans_v2.pdf, 2017. Lecture notes for STA 325, Chapter 10 of ISL, Duke University. 27

[37] Stanford University. Softmax regression. http://ufldl.stanford.edu/tutorial/supervised/SoftmaxRegression/, 2017. 44

[38] Amit K. Verma and A.D. (Tony) Rollett. Lecture 10 – random forest. Carnegie Mellon University, 2021.

[39] Peter Wiemer-Hastings, K Wiemer-Hastings, and A Graesser. Latent semantic analysis. In *Proceedings of the 16th international joint conference on Artificial intelligence*, pages 1–14, 2004. 20

[40] Paul Wohlhart, Martin Kostinger, Michael Donoser, Peter M Roth, and Horst Bischof. Optimizing 1-nearest prototype classifiers. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 460–467, 2013.

[41] Johannes Zimmermann, Markus Wolf, Astrid Bock, Doris Peham, and Cord Benecke. The way we refer to ourselves reflects how we relate to others: Associations between first-person pronoun use and interpersonal problems. *Journal of Research in Personality*, 47(3):218–225, 2013. 9